Models and Algorithms

Machine Learning Overview

Overview

★ AI and Machine Learning

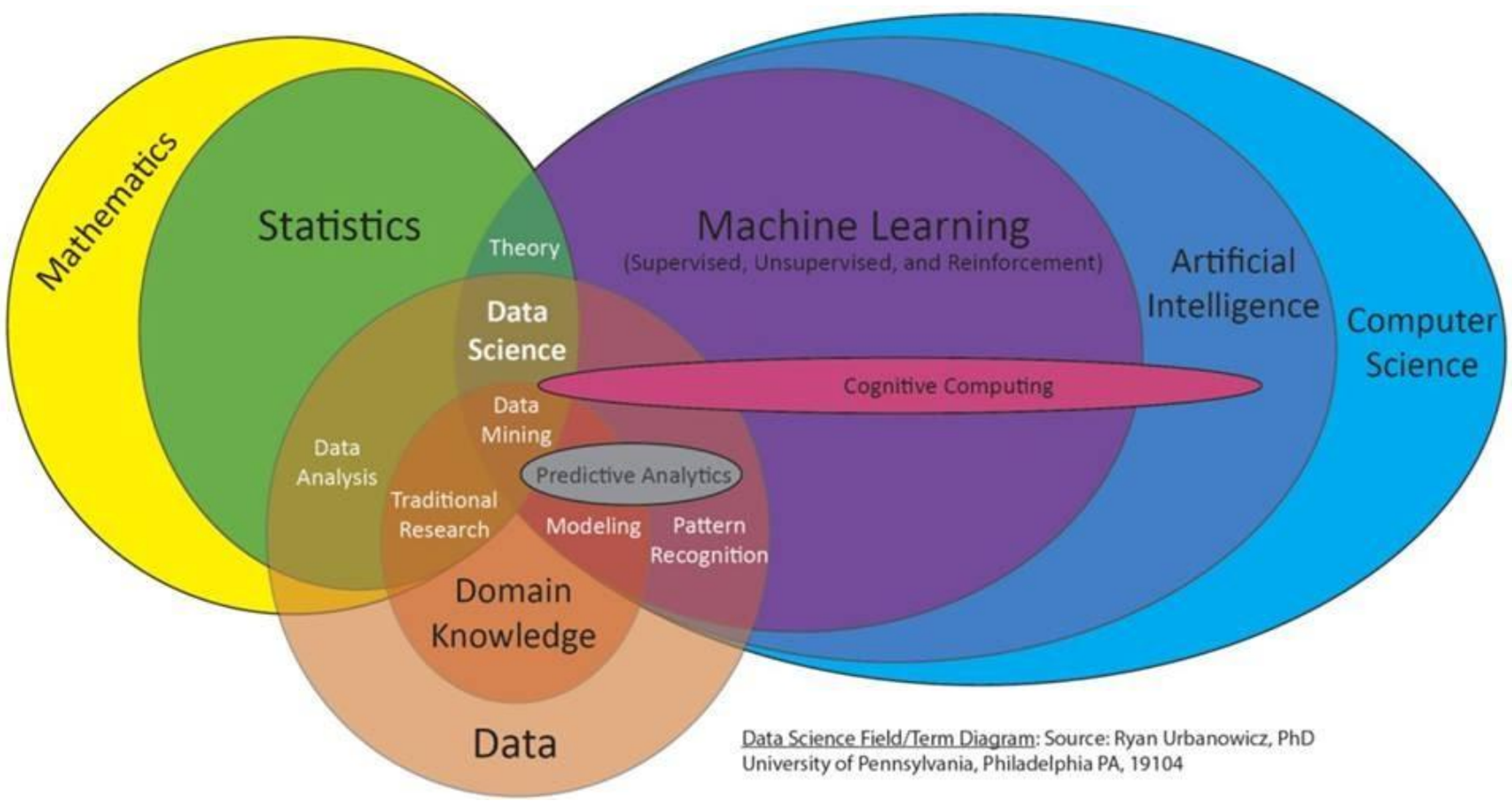
★ Machine Learning Scope: Data, Task, Model, and Algorithm

★ Data in Machine Learning

★ Machine Learning Tasks

Artificial Intelligence and Machine Learning

- Artificial Intelligence is the broader concept of machines being able to carry out tasks in a way that we would consider “smart”.
- Machine Learning is a current application of AI based around the idea that giving machines access to data and let them learn for themselves.



Data Science Field/Term Diagram: Source: Ryan Urbanowicz, PhD
University of Pennsylvania, Philadelphia PA, 19104

Machine Learning

- science of getting computers to act without being explicitly programmed.

- a branch of artificial intelligence focuses on the development of algorithms and statistical models

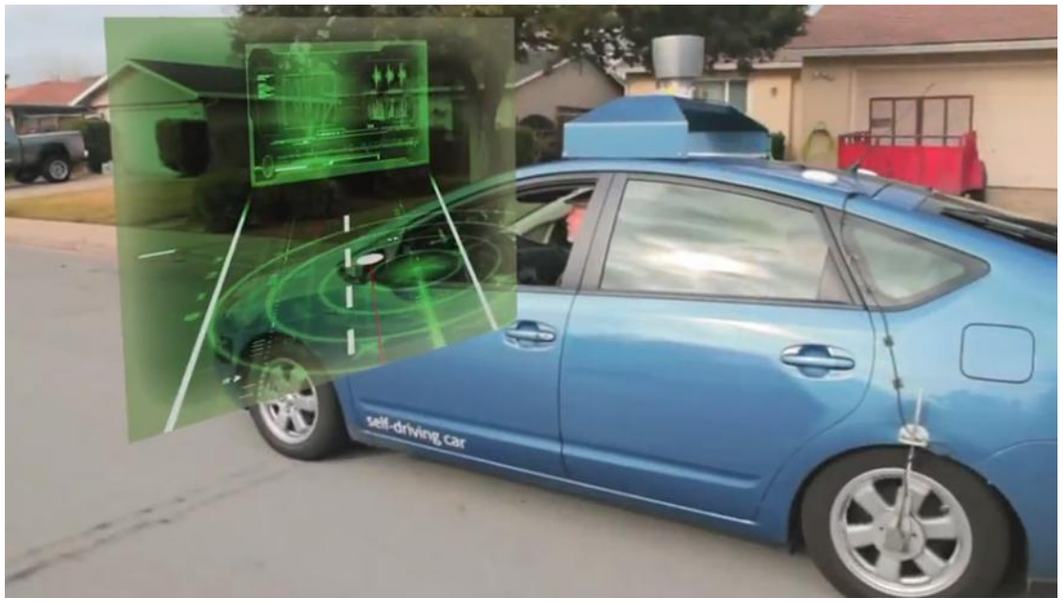
- based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.

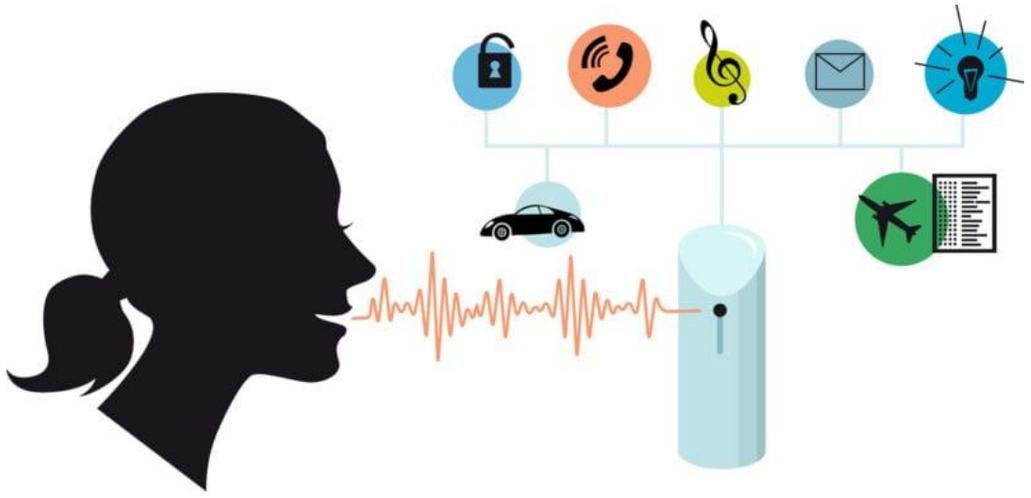
- a method of data analysis that automates analytical model building.

Machine Learning Applications

- In the past decade, machine learning has given us self-driving cars, speech recognition, effective web search, and a vastly improved understanding of the human genome

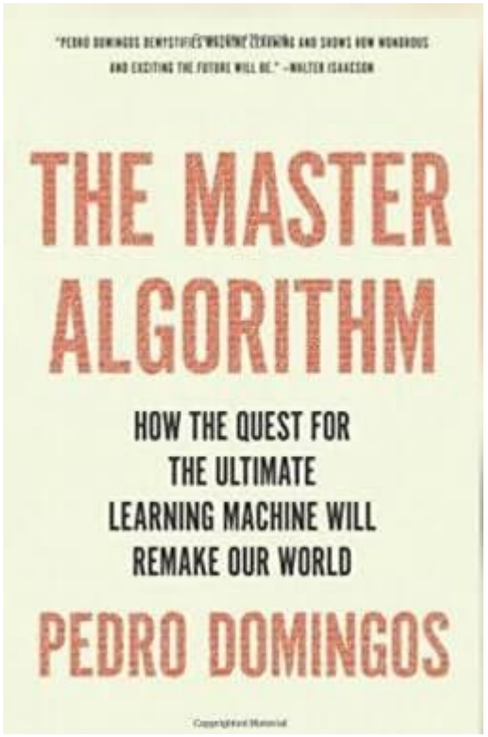
- Machine learning is so pervasive today that you probably use it dozens of times a day without knowing it.

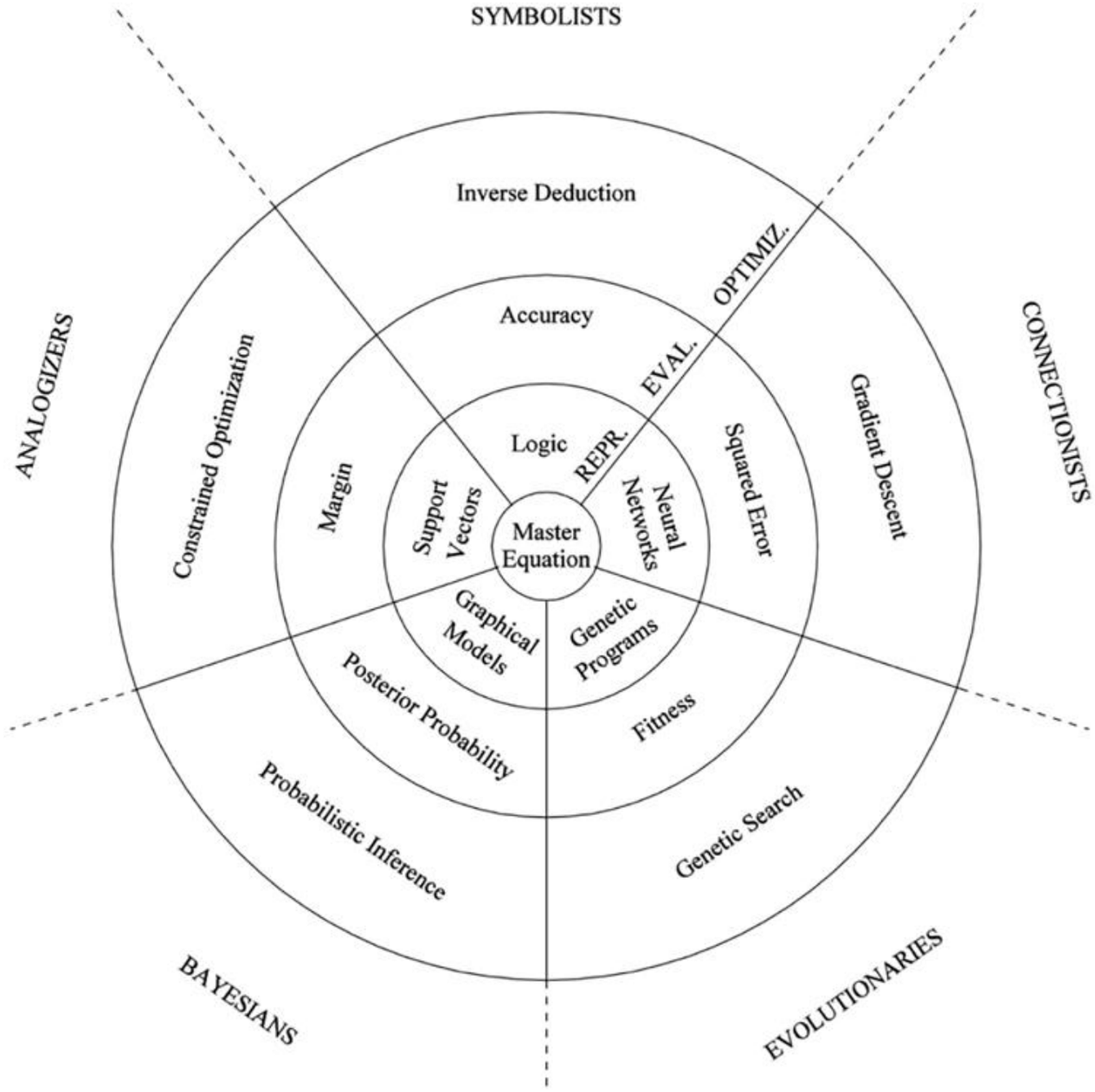




https://www.aiperspectives.com/speech-recognition/

Five Tribes of Machine Learning





Domingos, Pedro. "A few useful things to know about machine learning." *Communications of the ACM* 55, no. 10 (2012): 78-87.

The Scope of Machine Learning

ML involves a *wide variety* of each of these:

- data
- task
- model
- algorithm

The first 3 weeks --

- Today we'll address a couple of aspects of data.
- Next lecture: the tasks in Machine Learning
- Then: on to the most common models and algorithms (in overview only)

Data

- datahub.io
- openml.org

e.g.

- iris -- https://www.openml.org/d/61
- penguins -- https://www.openml.org/d/42585
- diabetes -- https://www.openml.org/d/37
- banknotes -- https://www.openml.org/d/1462
- …

Data

- eg: banknotes -- https://www.openml.org/d/1462
- someone having a go at clustering with this data: https://towardsdatascience.com/k-means-clustering-project-banknote-authentication-289cfe773873
- there are many blogs / tutorials / notebooks out there doing similar
- there are lots of new tools all the time, but near-generic tools at the moment:
  - python
  - numpy
  - sklearn
  - pandas
  - matplotlib
  - jupyter notebooks

Data

- Consider banknotes.csv:
- V1-V4 are values of 4 “features”
- the Class is 1 (legit) or 2 (forged)
- Common to talk separately about X and Y:

V1	V2	V3	V4	Class
3.6216	8.6661	-2.8073	-0.44699	1
4.5459	8.1674	-2.4586	-1.4621	1
3.866	-2.6383	1.9242	0.10645	1
3.4566	9.5228	4.0112	-3.5944	1
0.32924	-4.4552	4.5718	-0.9888	1
4.3684	9.6718	3.9606	-3.1625	1
3.5912	3.0129	0.72888	0.56421	1
2.0922	-6.81	8.4636	-0.60216	1
3.2032	5.7588	-0.75345	-0.61251	1
1.5356	9.1772	-2.2718	-0.73535	1
1.2247	8.7779	-2.2135	-0.80647	1

X

Y

~1300 rows in this case.
The “Class=2” ones are further down.

Data as “Vectors” in a “Space”

- each row is one data item, here consisting of a pairing:

V1	V2	V3	V4	Class
3.6216	8.6661	-2.8073	-0.44699	1

$$X \rightarrow y$$

- we might talk about $\mathbf{X} = (x_0, x_1, x_2, x_3)$ instead of the names v_1, etc specific to this dataset.
- ***X*** is a 4-dimensional *vector*
- x_i is the value for the ith *dimension*

Data as points in a space

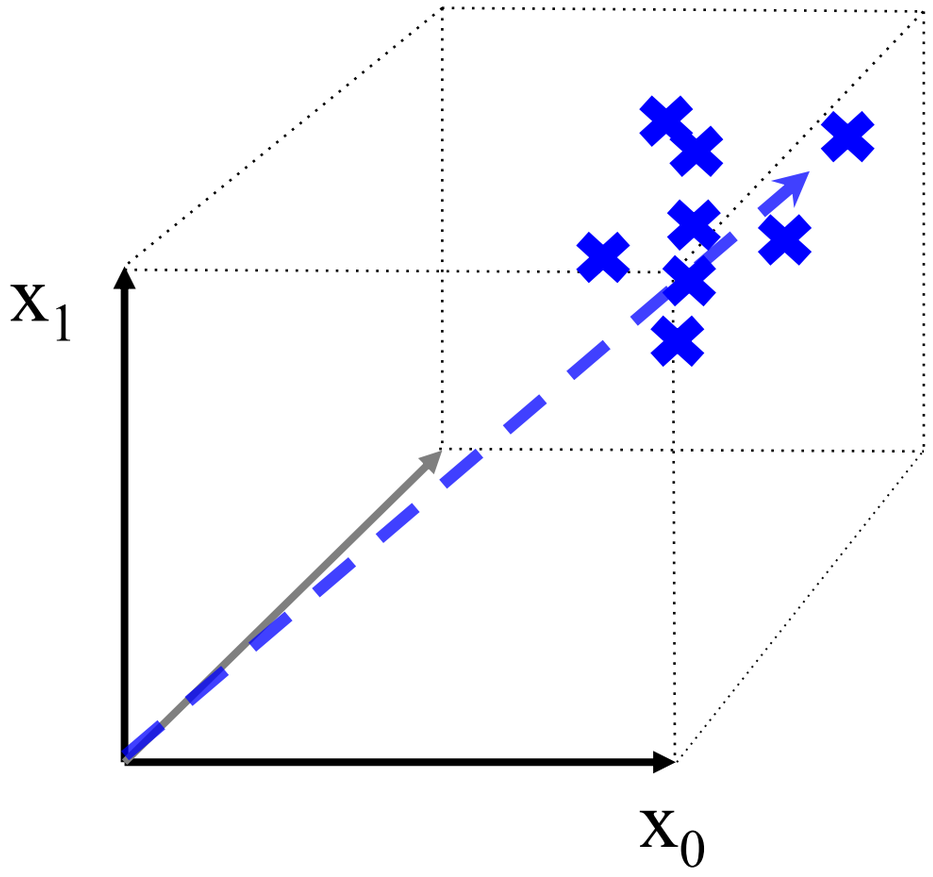
A “row” can be thought of as a “point” in a “space” of data

It’s easy to visualise when dimensionality is low:

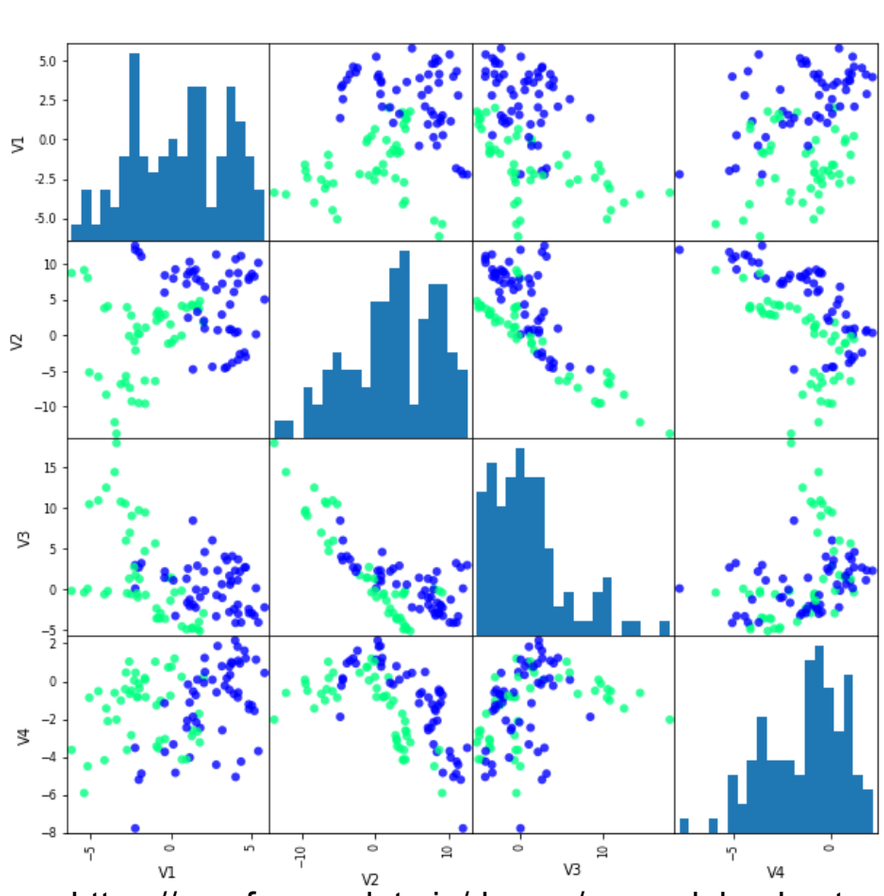
- 1 dimension, e.g. just V1
- 2 dimensions, easy
- 3 dimensions, easy

V1	V2	V3	V4	Class
3.6216	8.6661	-2.8073	-0.44699	1
4.5459	8.1674	-2.4586	-1.4621	1
3.866	-2.6383	1.9242	0.10645	1
3.4566	9.5228	-4.0112	-3.5944	1
0.32924	-4.4552	4.5718	-0.9888	1
4.3684	9.6718	-3.9606	-3.1625	1
3.5912	3.0129	0.72888	0.56421	1
2.0922	-6.81	8.4636	-0.60216	1
3.2032	5.7588	-0.75345	-0.61251	1
1.5356	9.1772	-2.2718	-0.73535	1
1.2247	8.7779	-2.2135	-0.80647	1

Data as Points in a Vector Space



V1	V2	V3	V4	Class
3.6216	8.6661	-2.8073	-0.44699	1
4.5459	8.1674	-2.4586	-1.4621	1
3.866	-2.6383	1.9242	0.10645	1
3.4566	9.5228	-4.0112	-3.5944	1
0.32924	-4.4552	4.5718	-0.9888	1
4.3684	9.6718	-3.9606	-3.1625	1
3.5912	3.0129	0.72888	0.56421	1
2.0922	-6.81	8.4636	-0.60216	1
3.2032	5.7588	-0.75345	-0.61251	1
1.5356	9.1772	-2.2718	-0.73535	1
1.2247	8.7779	-2.2135	-0.80647	1



https://sporf.neurodata.io/demos/openml_banknote

Humans can’t see in more than 3d

Some of our intuitions hold, some fail

Data in High dimensions

roughly speaking,

- it's all "in the corners"
- it's all "at right-angles"
- it's all "equally far apart"

```
1 for dims in range(1,21):
2     print(dims, countInside(1000000,dims))
```

```
1 1000000
2 785776
3 523488
4 307377
5 164873
6 80816
7 37145
8 15879
9 6370
10 2581
11 915
12 324
13 113
14 41
15 11
16 3
17 1
18 0
19 0
20 0
```

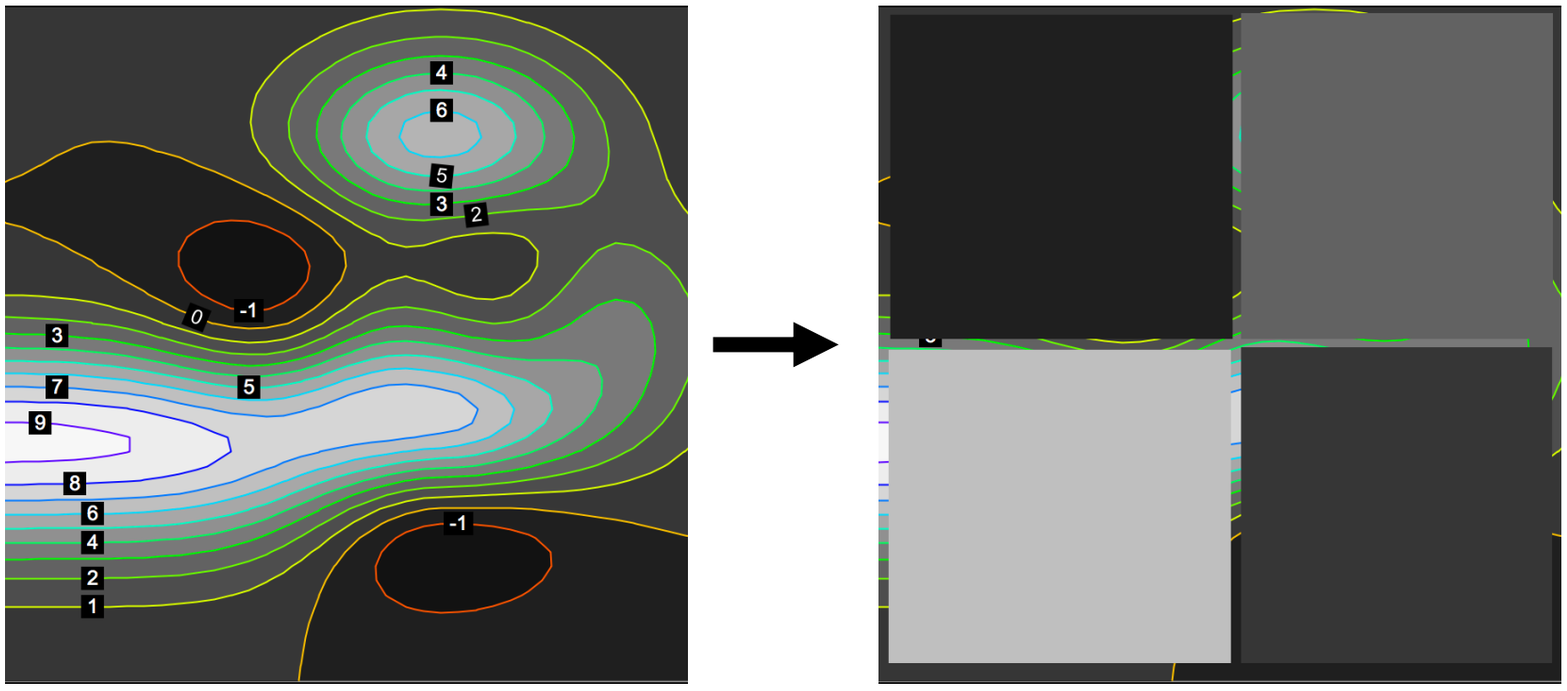
none

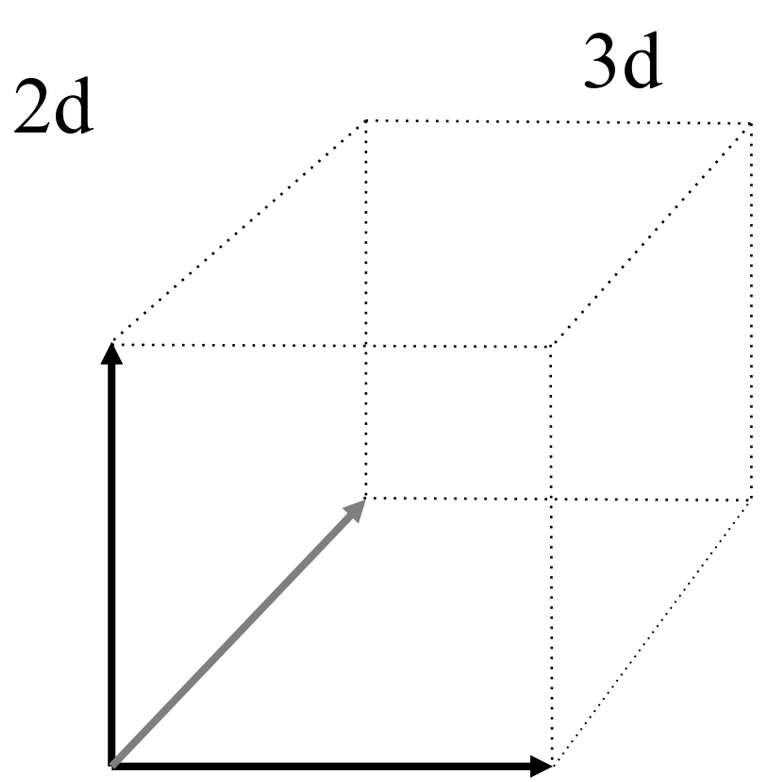
Eg: I made a million data points that were *d*-dimensional, with each dimension being randomly chosen in range -1 to +1. i.e. in a "box" of side 2.

A ball with diameter 2 fits snugly inside this box. Out of a million random points, how many land inside the ball?

The Curse of Dimensionality

- consider a vector space that is bounded within 0 to 1 for each dimension.
- I decide to just brutally summarise / compress the function, by giving just *one value per "quadrant". Eg. in 2d, I need 4 numbers.*





- *how many numbers do I need, in d-dimensions?*

Lecture 05 – Models and Algorithms

Machine Learning Tasks

Overview

★ AI and Machine Learning

★ Machine Learning Scope: Data, Task, Model, and Algorithm

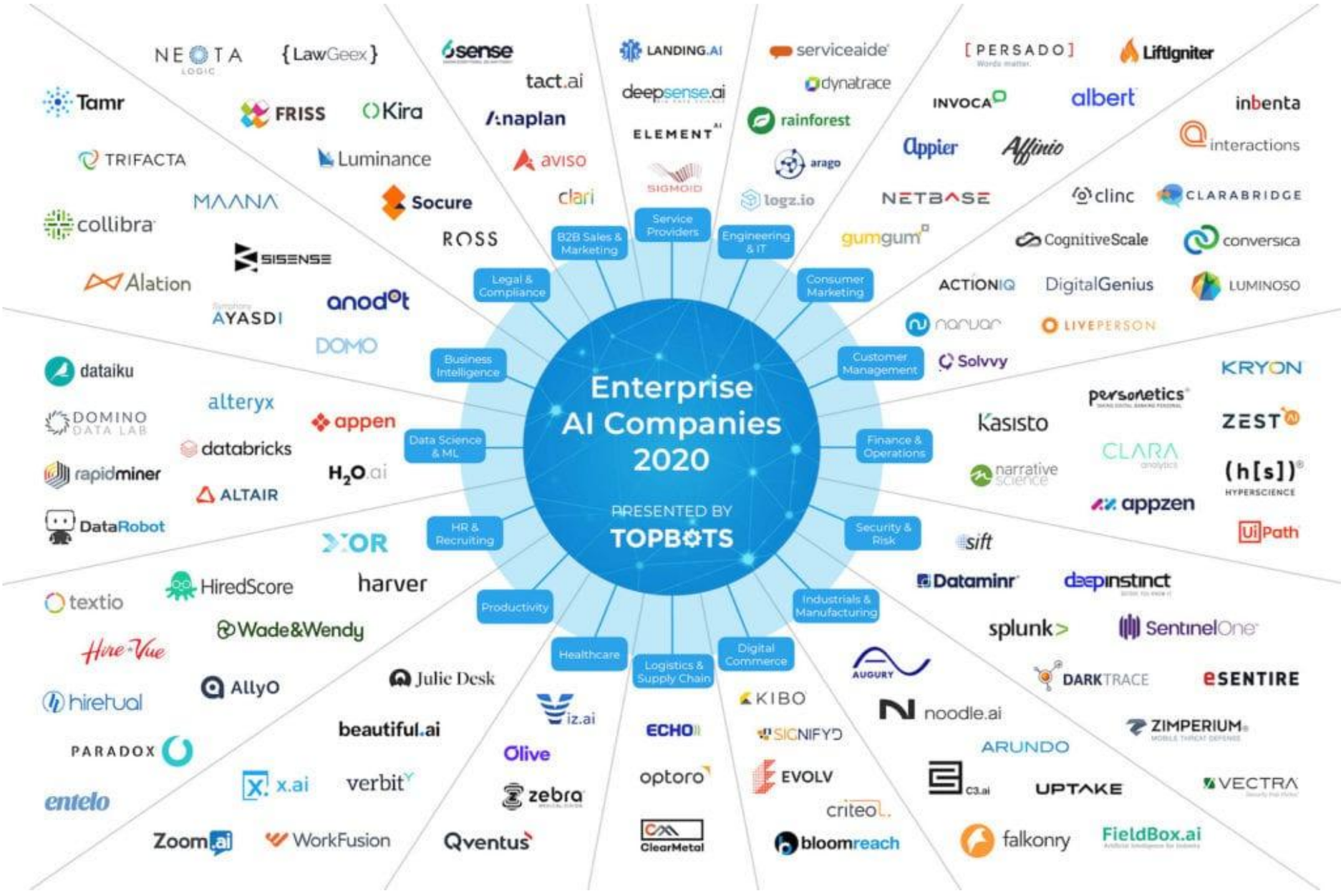
★ Data handle by Machine Learning

★ Machine Learning Tasks

Artificial Intelligence Companies



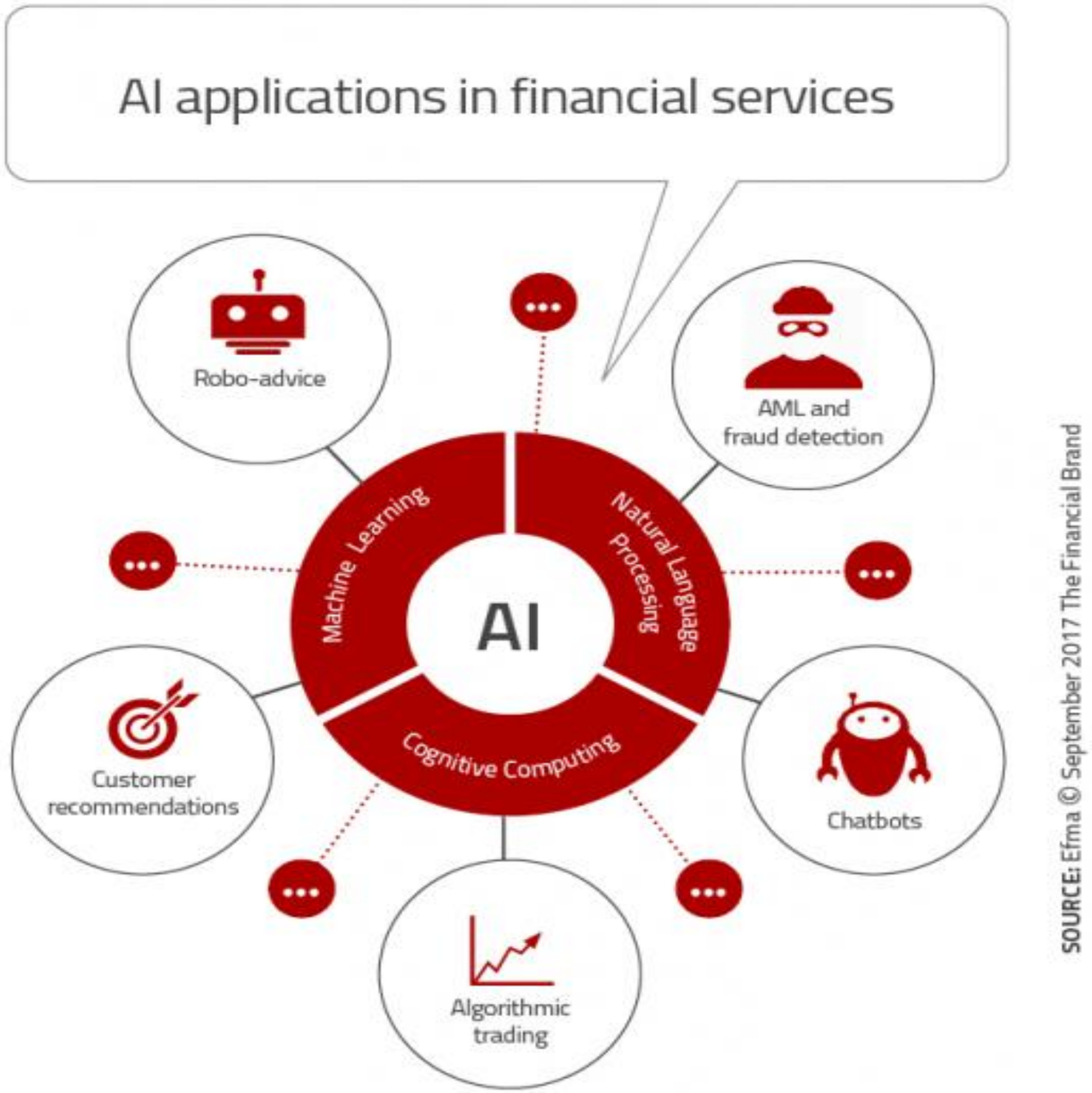
Artificial Intelligence Companies



cf in Aotearoa :

https://newzealand.ai/nz-ai-companies

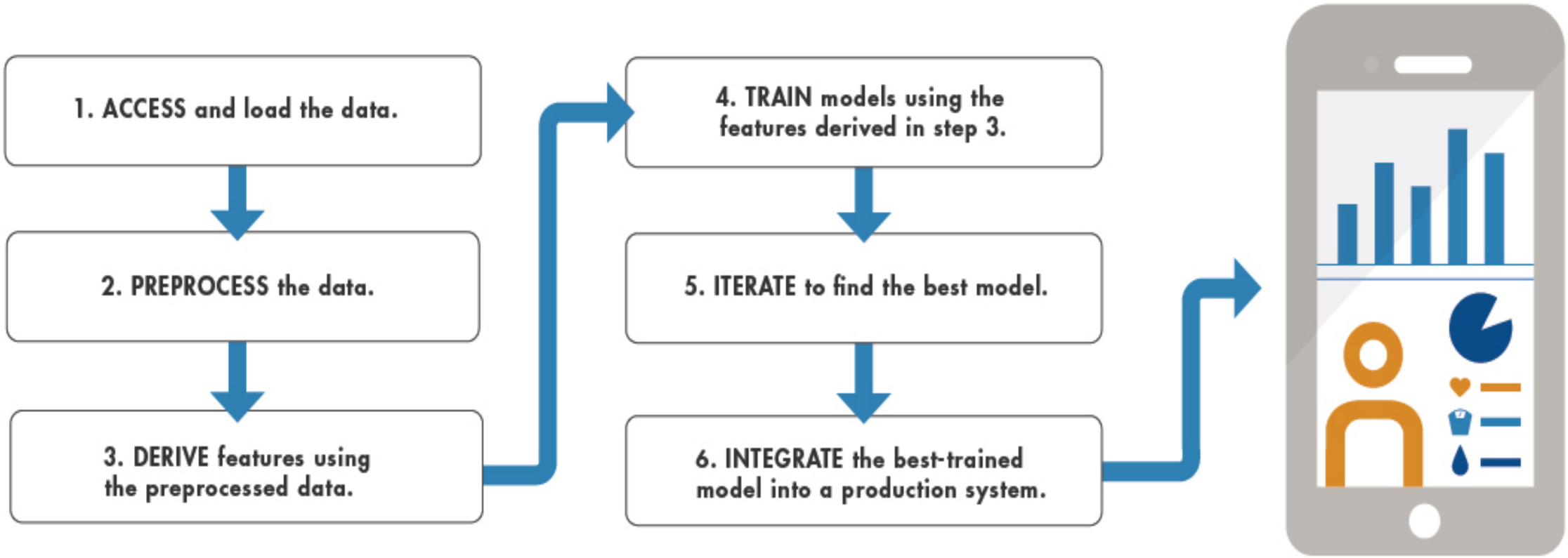
Not This - This is Application



Not this either - this is the process of ML

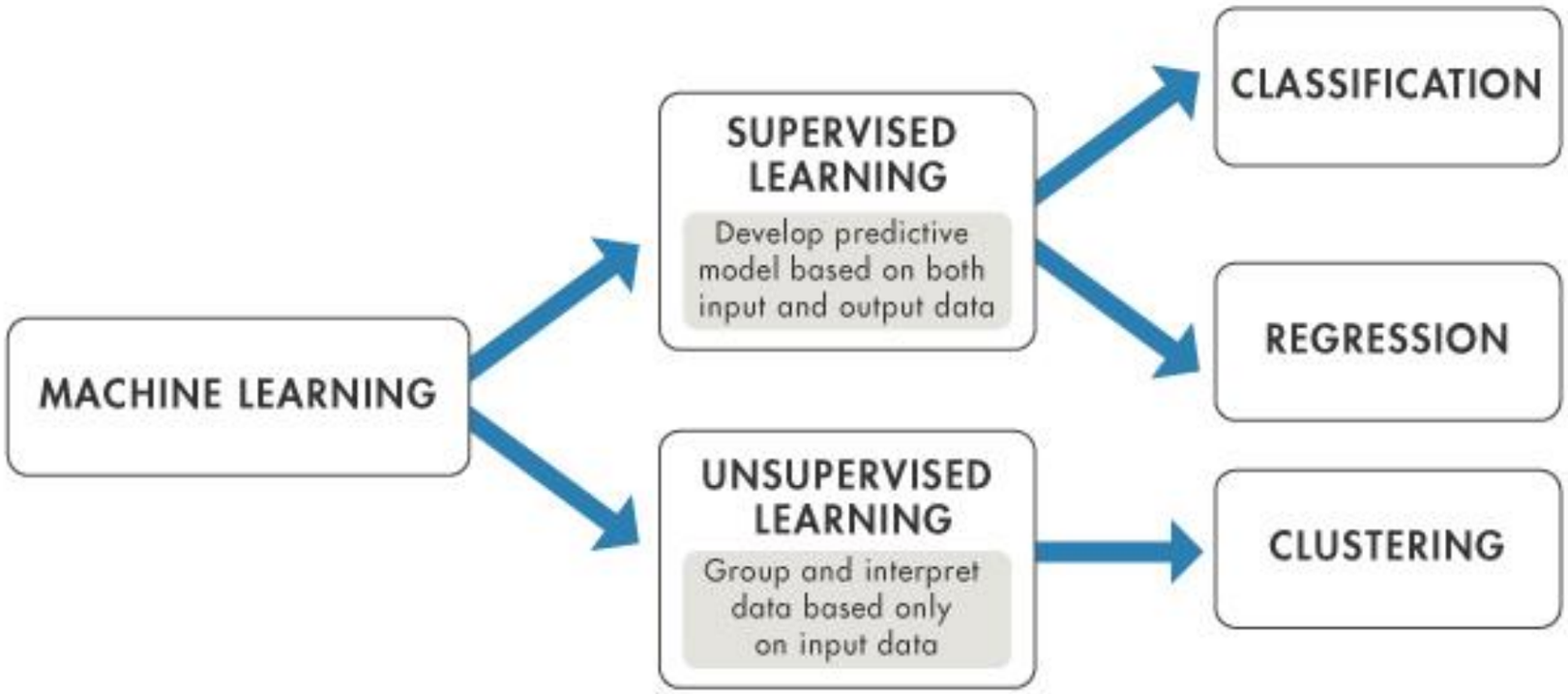


This is the process of ML



https://www.mathworks.com/help/stats/machine-learning-in-matlab.html

MATLAB gives the following ontology of ML



But it’s a lot richer than this

https://www.mathworks.com/help/stats/machine-learning-in-matlab.html

Supervision available to the learner

lots:

Supervised learning: The environment contains a teacher that provides the correct response for certain environmental states. The goal is for the learner to output the correct response: “do what the teacher would do”.

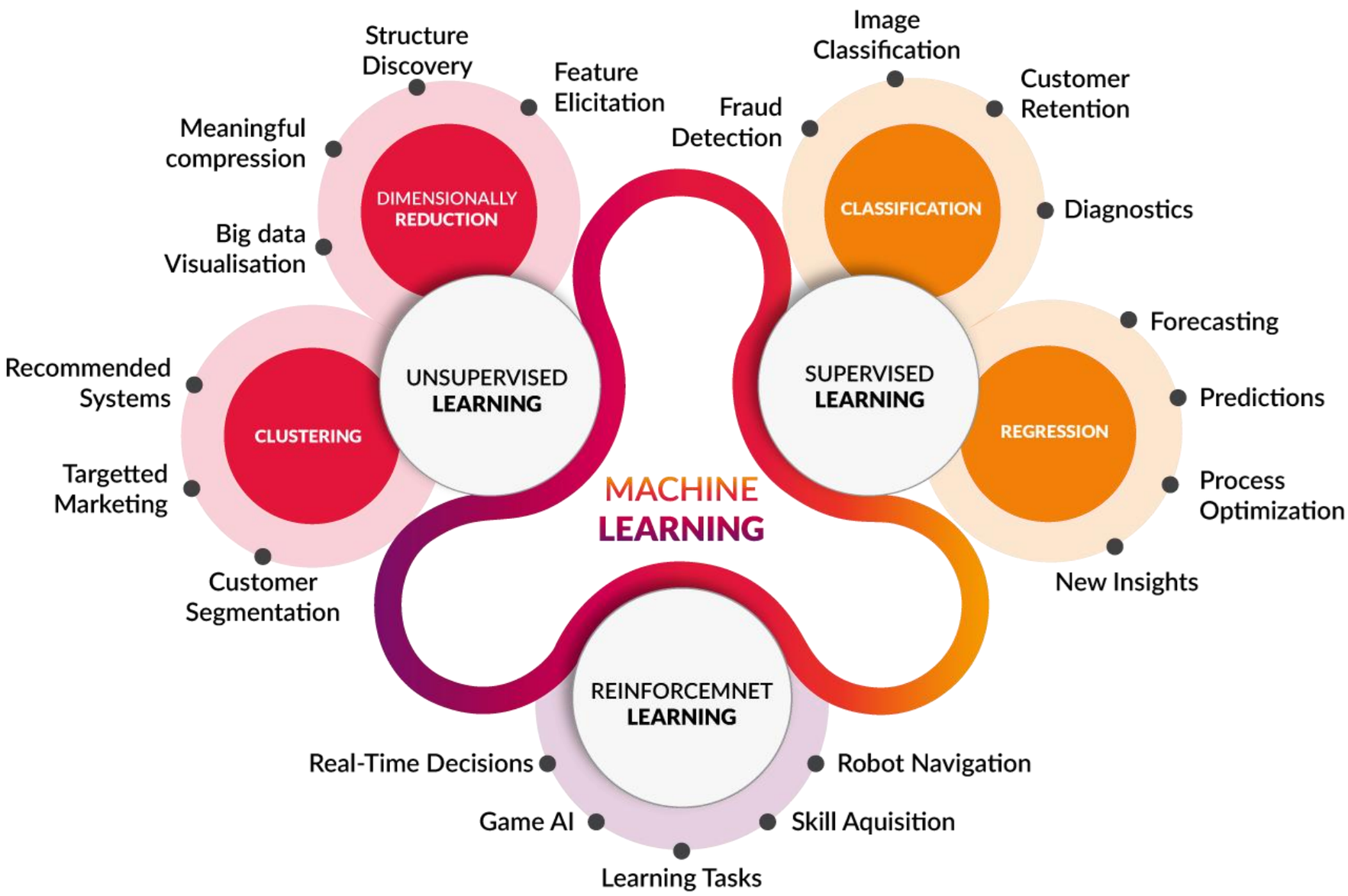
none:

Unsupervised learning: No such teacher to say whether the learner’s output is correct. Instead the learning system has an internally defined teacher with a prescribed goal that does not need utility feedback of any kind.

some:

Reinforcement learning: Again, no such teacher to say whether the learner’s output is correct. Instead of a label, the environment provides reward or punishment to indicate the utility of actions that were actually taken by the system.

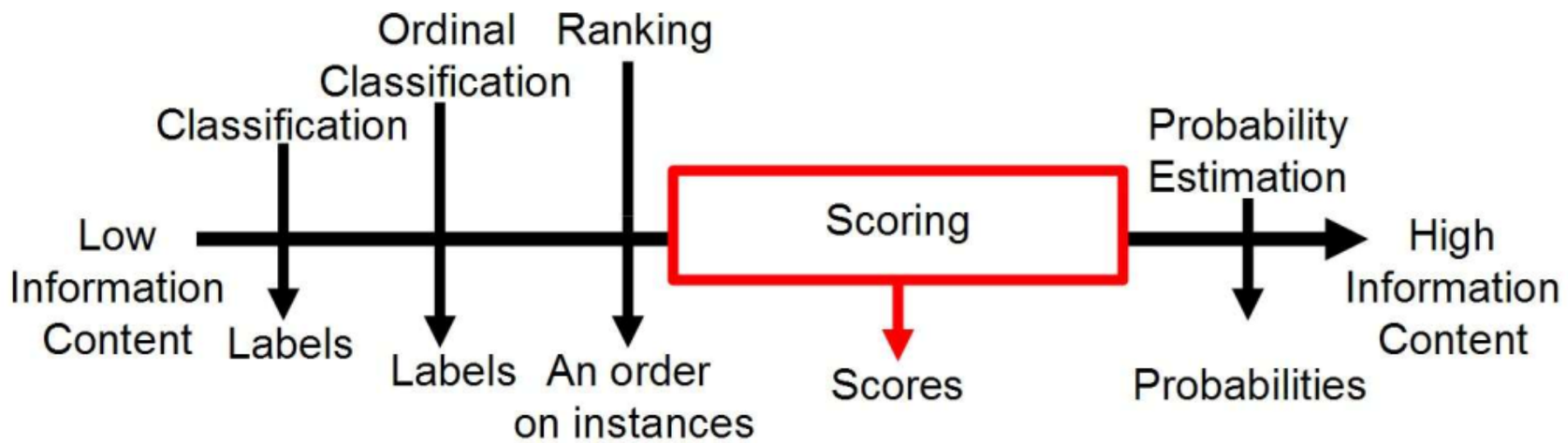
3-way split



http://www.cognub.com/index.php/cognitive-platform/

Outcomes (outputs) – can be simple… or more informative

- Notice that different outcomes can have different amounts of information content
- *e.g. here, a classifier classifies☺ some novel input pattern*

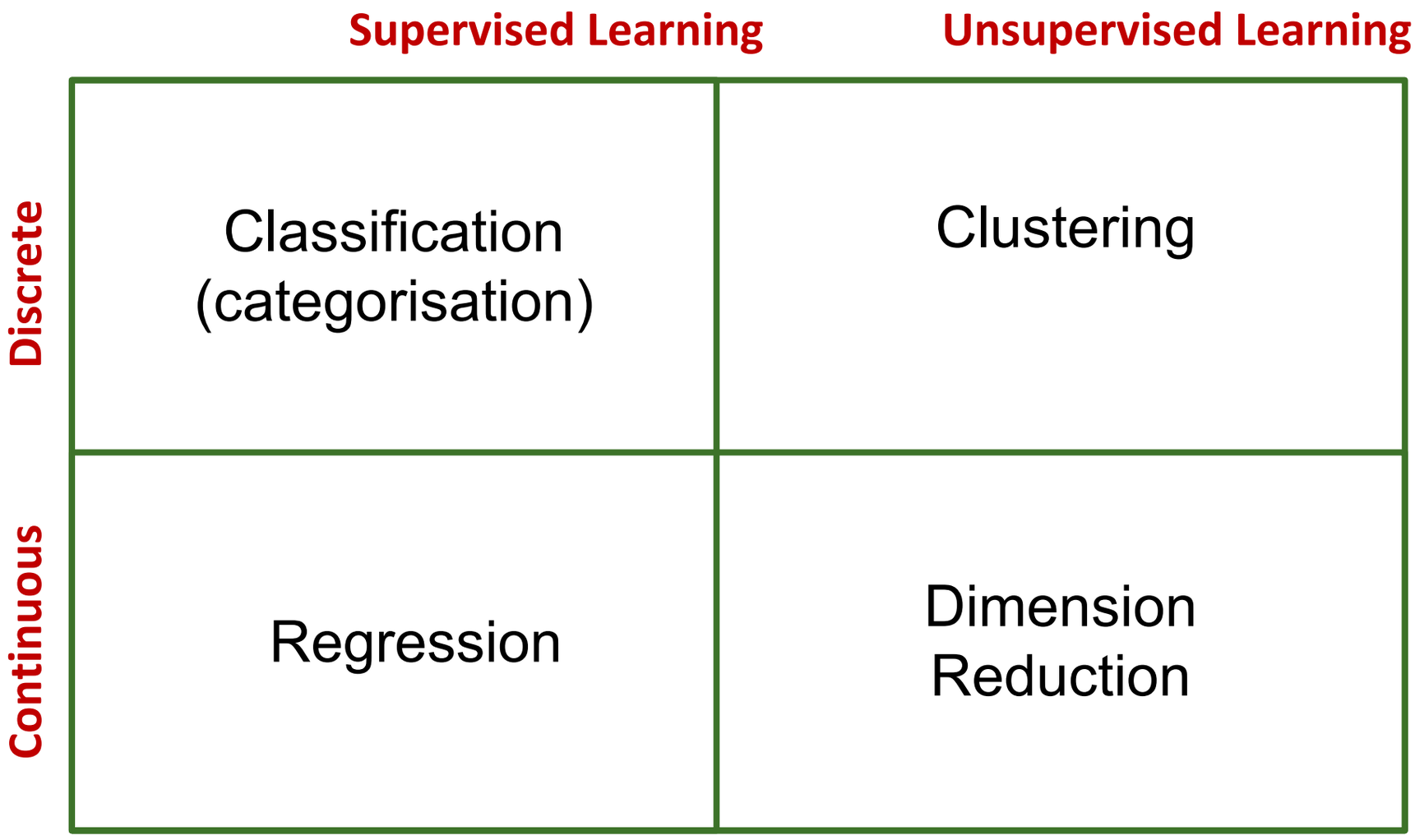


notice we can think of the classifier's response a "prediction", of a kind

Prediction Outcomes

https://www.researchgate.net/publication/220699539_Smooth_Receiver_Operating_Characteristics_smROC_Curves

The main divisions within learners



(leaving Reinforcement learning aside)

The main divisions within learners

	Supervised Learning	Unsupervised Learning
Discrete	Classification (categorisation)	Clustering
Continuous	Regression	Dimension Reduction

Semi-supervised Learning?

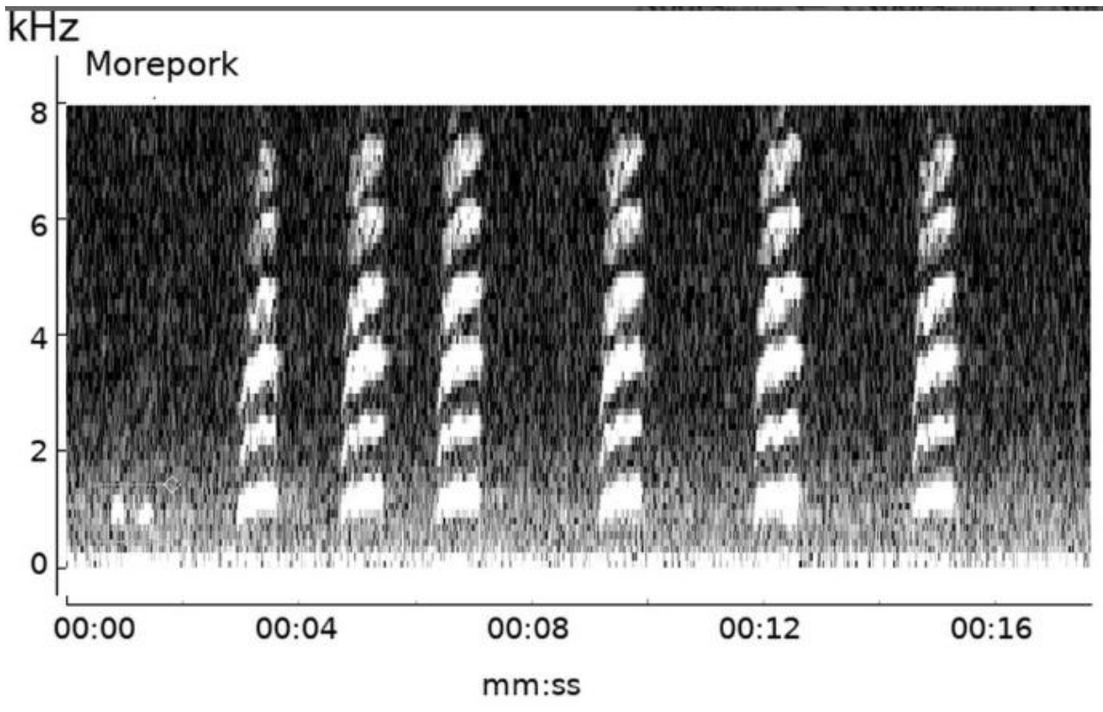
Adapted from: https://www.slideshare.net/GirishKhanzode/supervised-learning-52218215

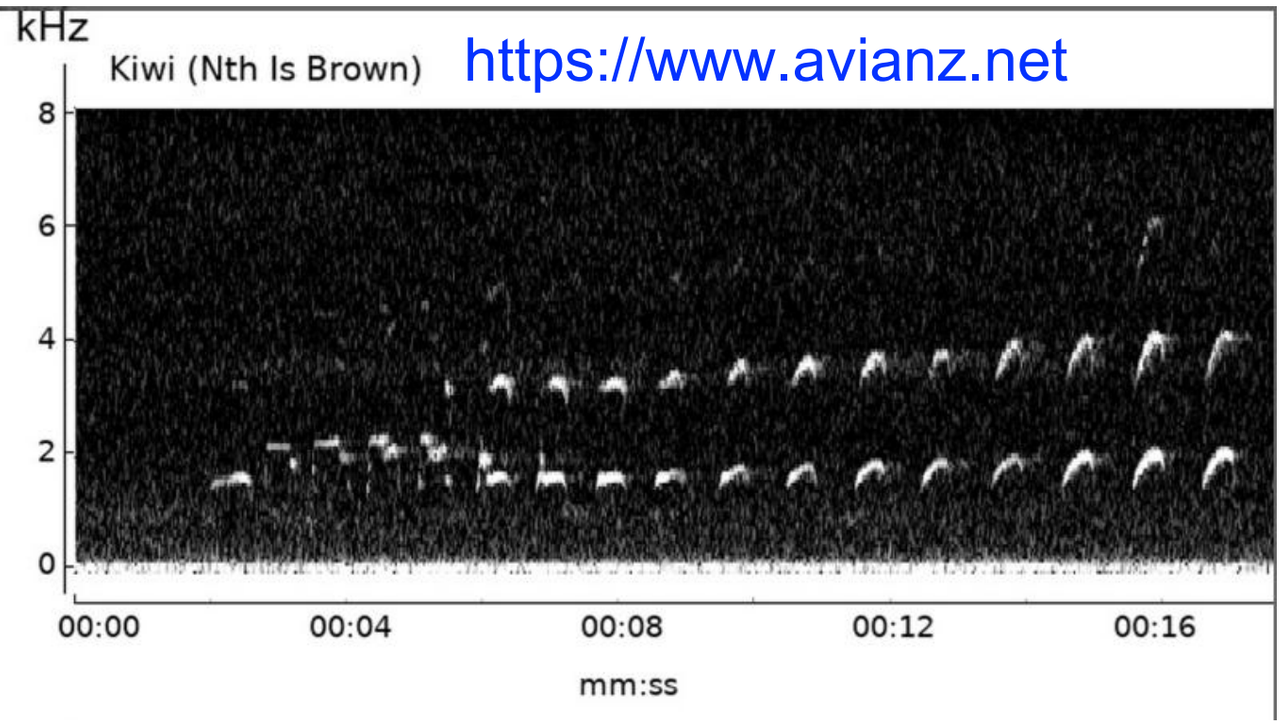
Types of Input

Could be…

- i.i.d. (**i**ndependent, **i**dentically **d**istributed)
- or a sequence, like this:

V1	V2	V3	V4	Class
3.6216	8.6661	-2.8073	-0.44699	1
4.5459	8.1674	-2.4586	-1.4621	1
3.866	-2.6383	1.9242	0.10645	1
3.4566	9.5228	-4.0112	-3.5944	1
0.32924	-4.4552	4.5718	-0.9888	1
4.3684	9.6718	-3.9606	-3.1625	1
3.5912	3.0129	0.72888	0.56421	1
2.0922	-6.81	8.4636	-0.60216	1
3.2032	5.7588	-0.75345	-0.61251	1
1.5356	9.1772	-2.2718	-0.73535	1
1.2247	8.7779	-2.2135	-0.80647	1





- or…?

Classification

- Predict a category(class)

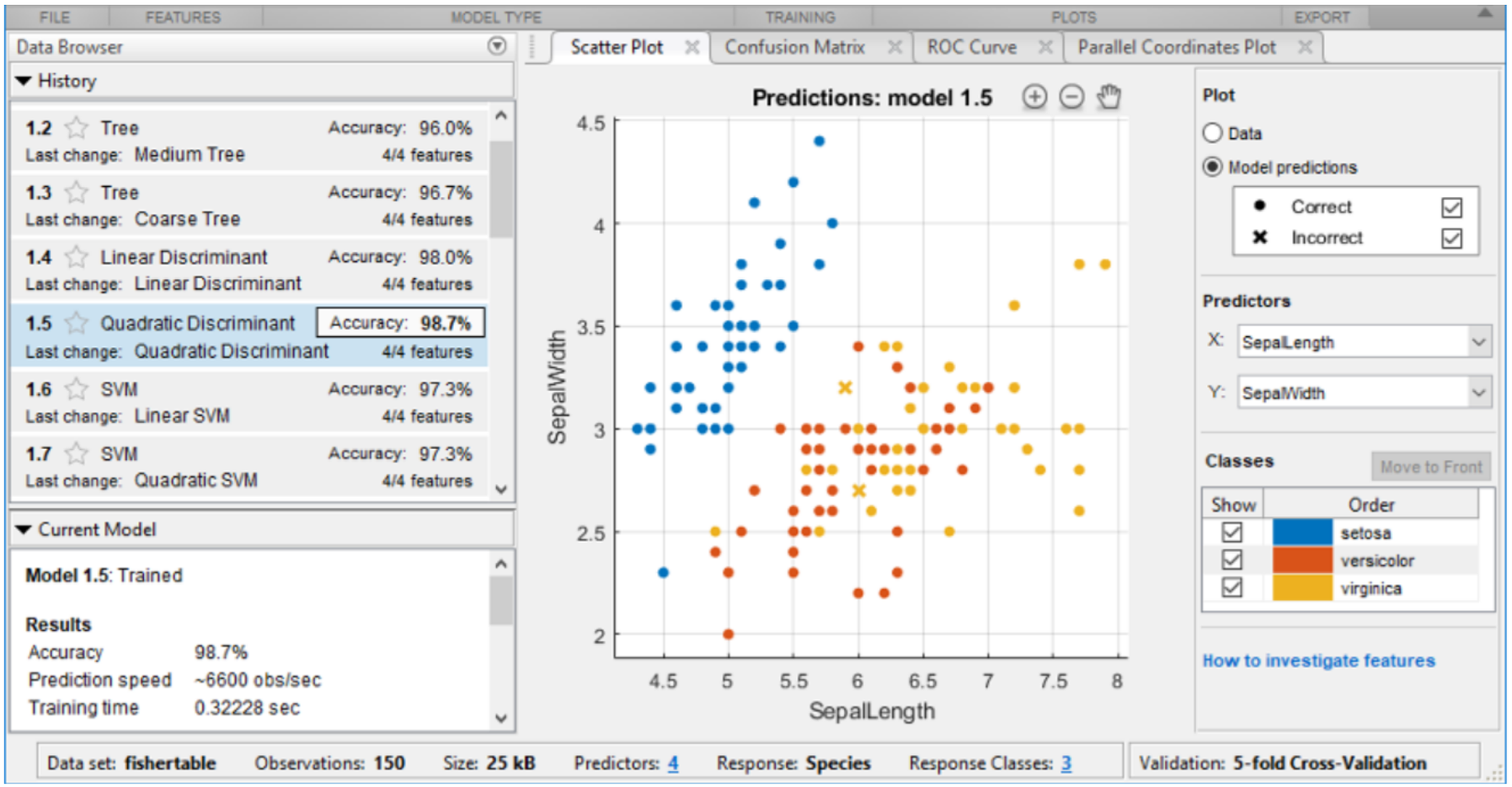
V1	V2	V3	V4	Class
3.6216	8.6661	-2.8073	-0.44699	1
4.5459	8.1674	-2.4586	-1.4621	1
3.866	-2.6383	1.9242	0.10645	1
3.4566	9.5228	-4.0112	-3.5944	1
0.32924	-4.4552	4.5718	-0.9888	1
4.3684	9.6718	-3.9606	-3.1625	1
3.5912	3.0129	0.72888	0.56421	1
2.0922	-6.81	8.4636	-0.60216	1
3.2032	5.7588	-0.75345	-0.61251	1
1.5356	9.1772	-2.2718	-0.73535	1
1.2247	8.7779	-2.2135	-0.80647	1

species of iris: (a 3-way classification)

	Sepal length	Sepal width	Petal length	Petal width	Type
1	5.1	3.5	1.4	0.2	Iris setosa
2	4.9	3.0	1.4	0.2	Iris setosa
…					
51	7.0	3.2	4.7	1.4	Iris versicolor
52	6.4	3.2	4.5	1.5	Iris versicolor
…					
101	6.3	3.3	6.0	2.5	Iris virginica
102	5.8	2.7	5.1	1.9	Iris virginica
…					

Classification

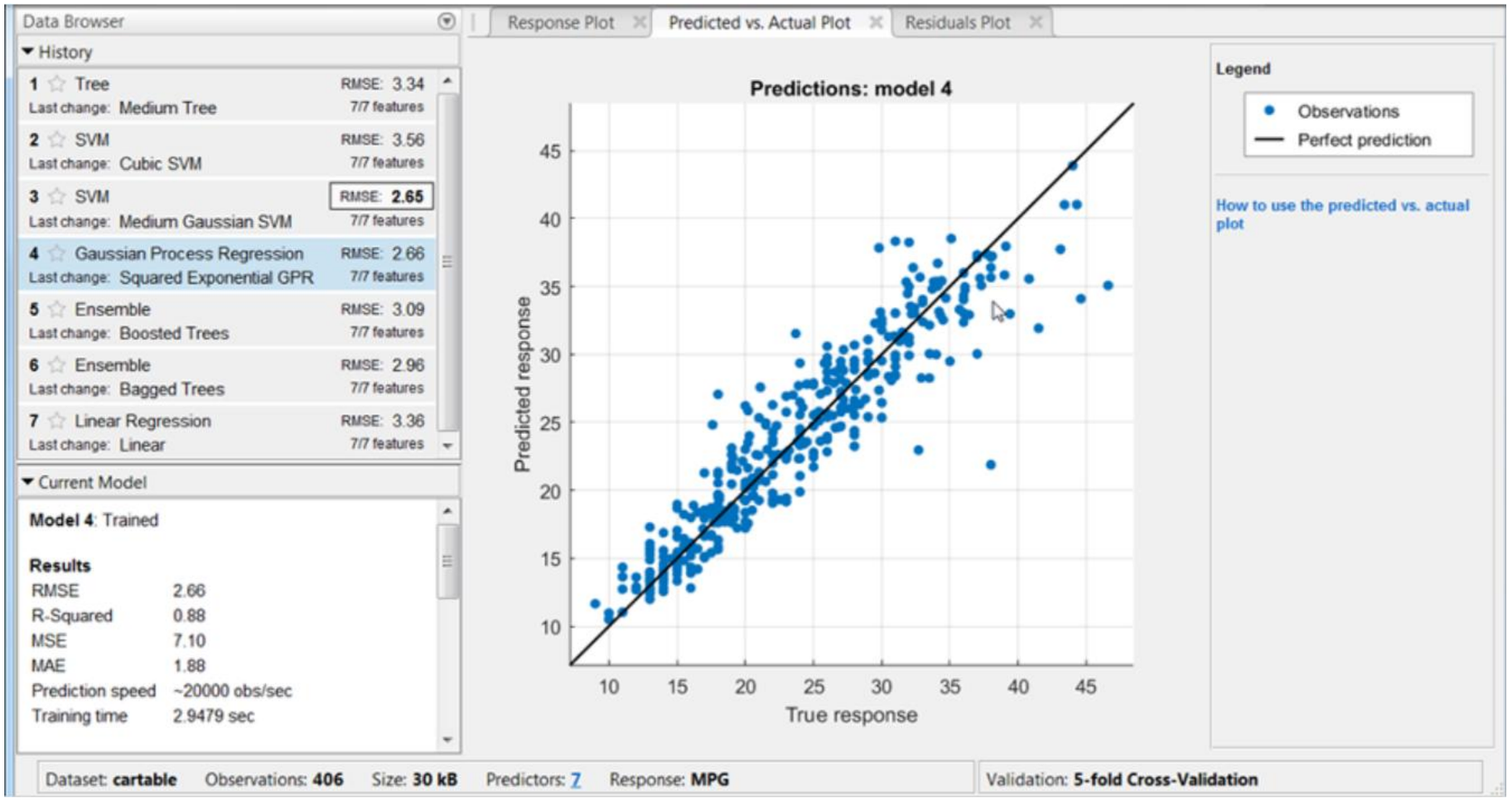
species of iris:
(a 3-way
classification)



https://www.mathworks.com/help/stats/machine-learning-in-matlab.html

Regression

- Predict one or more floats



Clustering

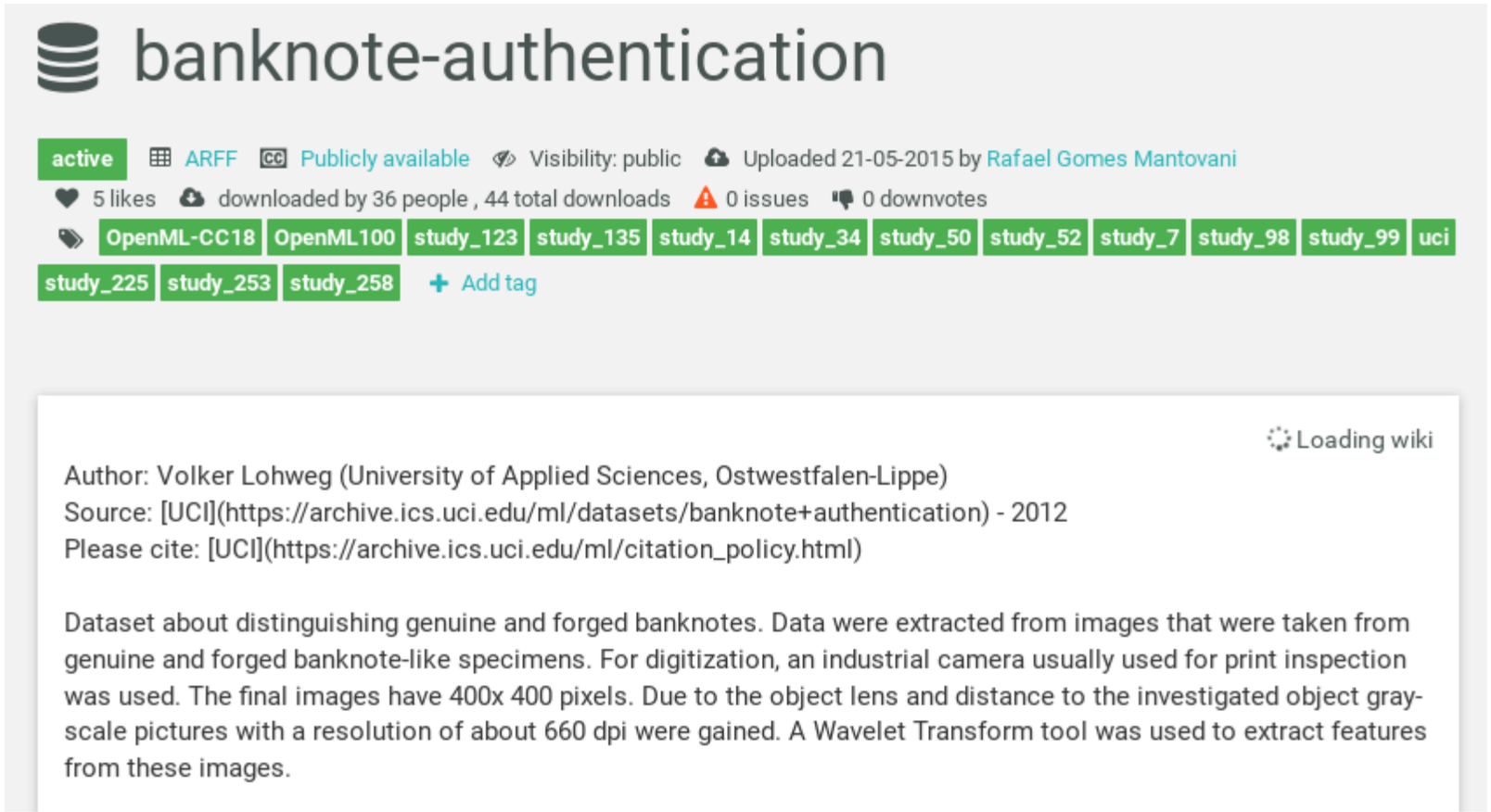
- Finding groups of items that are “similar”
- Clustering is *unsupervised:* class of an example is not known
- Success often measured subjectively – it is <u>fundamentally ill-posed</u>!

	Sepal length	Sepal width	Petal length	Petal width	Type
1	5.1	3.5	1.4	0.2	Iris setosa
2	4.9	3.0	1.4	0.2	Iris setosa
…					
51	7.0	3.2	4.7	1.4	Iris versicolor
52	6.4	3.2	4.5	1.5	Iris versicolor
…					
101	6.3	3.3	6.0	2.5	Iris virginica
102	5.8	2.7	5.1	1.9	Iris virginica
…					

Dimension Reduction

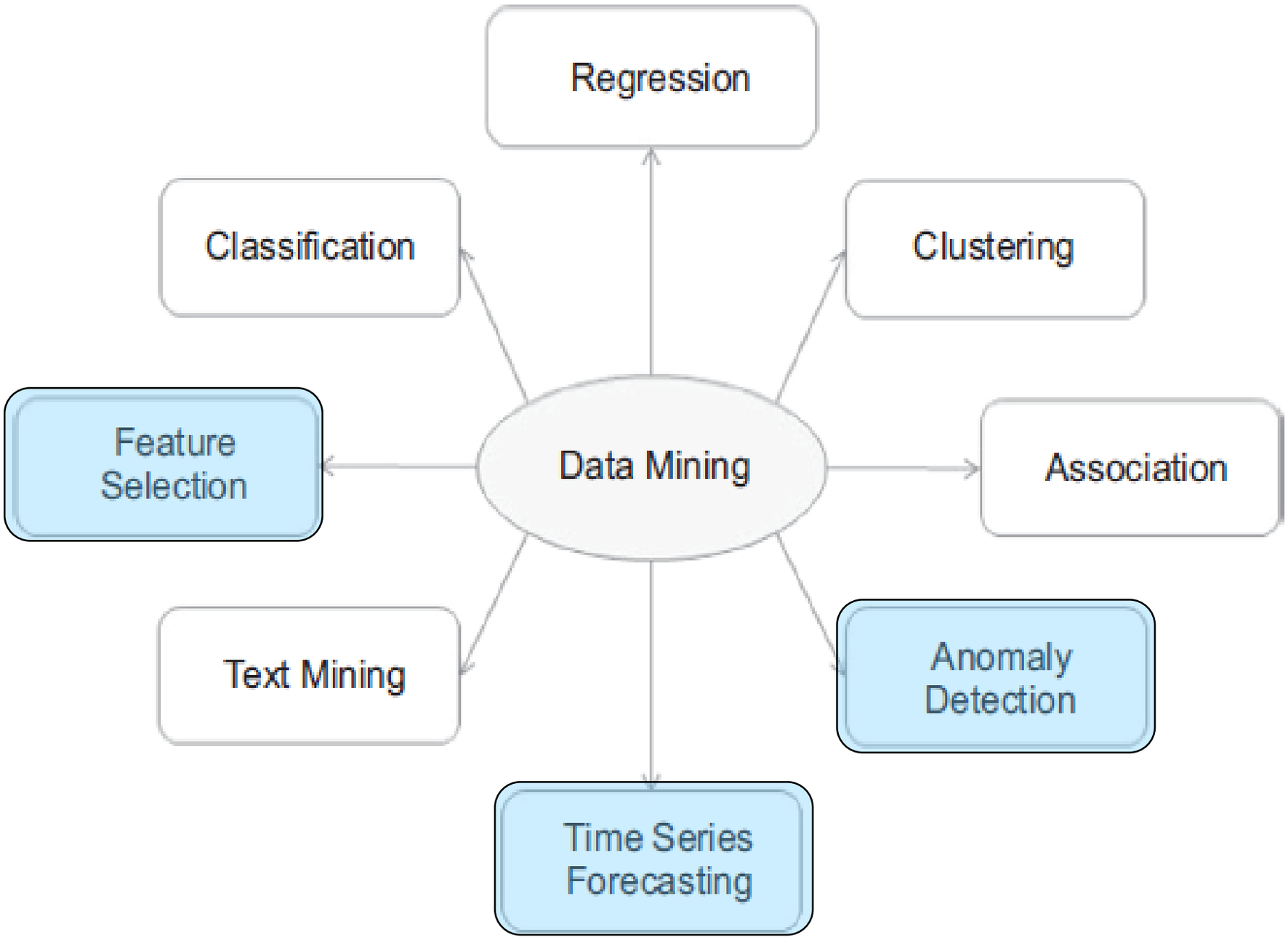
Raw data is often high-dimensional, which is a problem:

1. data with >3 dimensions is hard for humans to interpret / work with!
2. ML algorithms also struggle with high-dimensional data (ultimately, due to the curse of dimensionality)



Hence, interest in ML methods that can identify the main directions of interest in data, for example (e.g. PCA: Principle Components Analysis, and others)

Some Others


Regression
Classification
Clustering
Feature Selection
Data Mining
Association
Text Mining
Anomaly Detection
Time Series Forecasting

Some Others

Tasks	Description	Algorithms	Examples
Classification	Predict if a data point belongs to one of the predefined classes. The prediction will be based on learning from a known data set.	Decision trees, neural networks, Bayesian models, induction rules, k-nearest neighbors	Assigning voters into known buckets by political parties, e.g., soccer moms Bucketing new customers into one of the known customer groups
Regression	Predict the numeric target label of a data point. The prediction will be based on learning from a known data set.	Linear regression, logistic regression	Predicting unemployment rate for next year Estimating insurance premium
Anomaly detection	Predict if a data point is an outlier compared to other data points in the data set.	Distance based, density based, local outlier factor (LOF)	Fraud transaction detection in credit cards Network intrusion detection
Time series	Predict the value of the target variable for a future time frame based on historical values.	Exponential smoothing, autoregressive integrated moving average (ARIMA), regression	Sales forecasting, production forecasting, virtually any growth phenomenon that needs to be extrapolated
Clustering	Identify natural clusters within the data set based on inherit properties within the data set.	k-means, density-based clustering (e.g., density-based spatial clustering of applications with noise [DBSCAN])	Finding customer segments in a company based on transaction, web, and customer call data
Association analysis	Identify relationships within an item set based on transaction data.	Frequent Pattern Growth (FP-Growth) algorithm, Apriori algorithm	Find cross-selling opportunities for a retailer based on transaction purchase history

https://tanthiamhuat.wordpress.com/data-mining/

Lecture 05 – Models and Algorithms

Classification Tasks

Overview

★ Machine Learning Models and Algorithms

★ Classification: purpose, definition, different aspects

★ Popular Classifiers: KNN, Perceptrons, Decision Trees, SVM, Boosting, Random Forests

The Scope of ML

ML involves a *wide variety* of each of these:

- data
- task
- **model**
- **algorithm**

used interchangeably (and we will too 😐) but:

model : the theoretical basis on which a classification is made

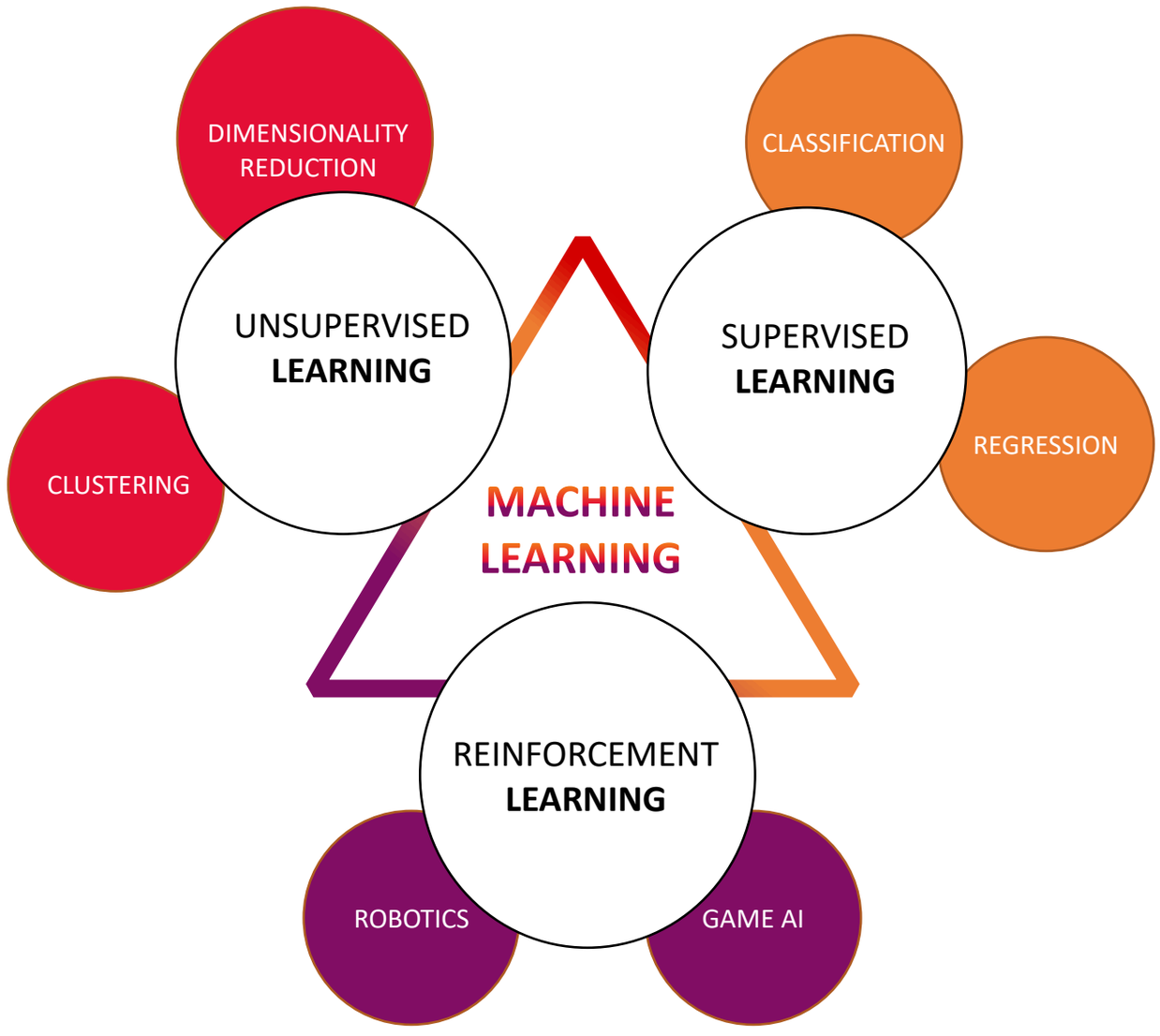
algorithm : the particular way we did it

did what? Distinguish:

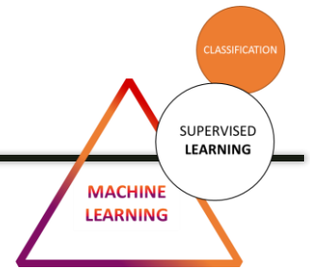
1. how a model gets *learned*
2. how a model gets *used*

These get conflated, because (2) involved in (1)

Tasks


DIMENSIONALITY REDUCTION
CLASSIFICATION
UNSUPERVISED LEARNING
SUPERVISED LEARNING
CLUSTERING
REGRESSION
MACHINE LEARNING
REINFORCEMENT LEARNING
ROBOTICS
GAME AI

Classification – Two Purposes



Prediction:

Given past data, construct a model such that a new observation (without its associated category) can be classified.

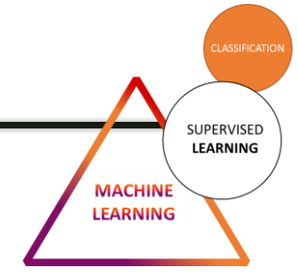
- Estimate the worth of the model, e.g. accuracy
- *Test set* independent of the training set (else overfitting)
- If a dataset is used to select/tune models, it is called a *Validation set*

Insight:

Identify any redundant/irrelevant features, feature importance and feature relationships to provide insight into the object.

- Observations (examples, samples, tuples) in the *training set* are used for model construction
- Models can be rules, decision trees, networks, ...

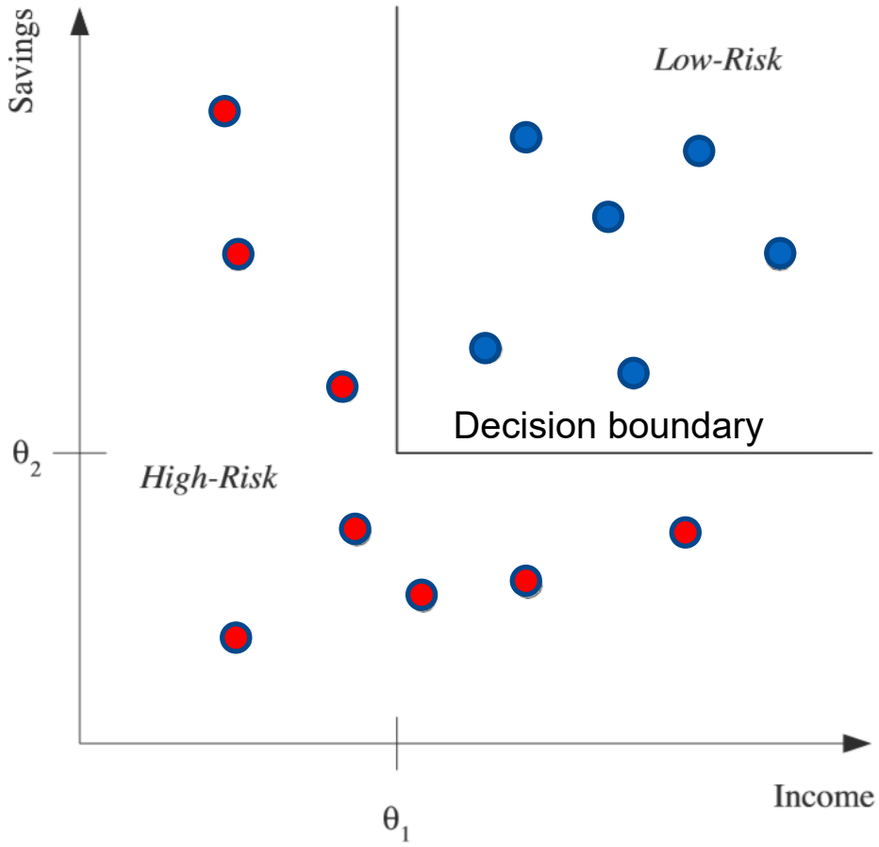
Classification – Definition



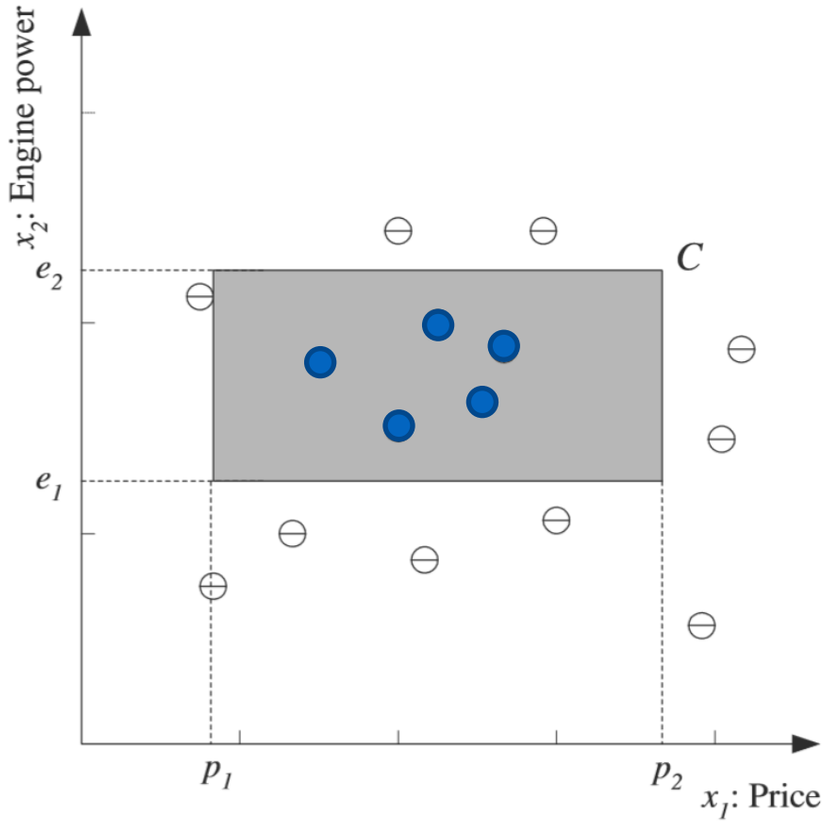
Search space *S* is the set of all possible instances.

Class *C* that needs to modelled.

Insurance

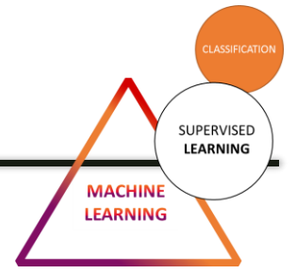


Car Sales



https://mitpress.mit.edu/books/introduction-machine-learning

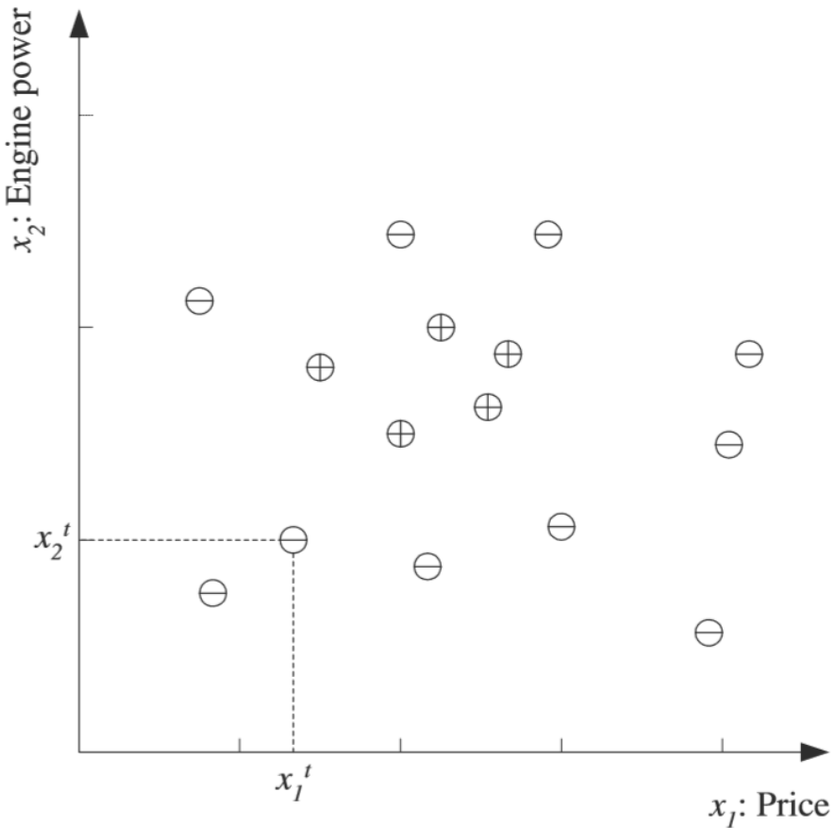
Aspects of classification



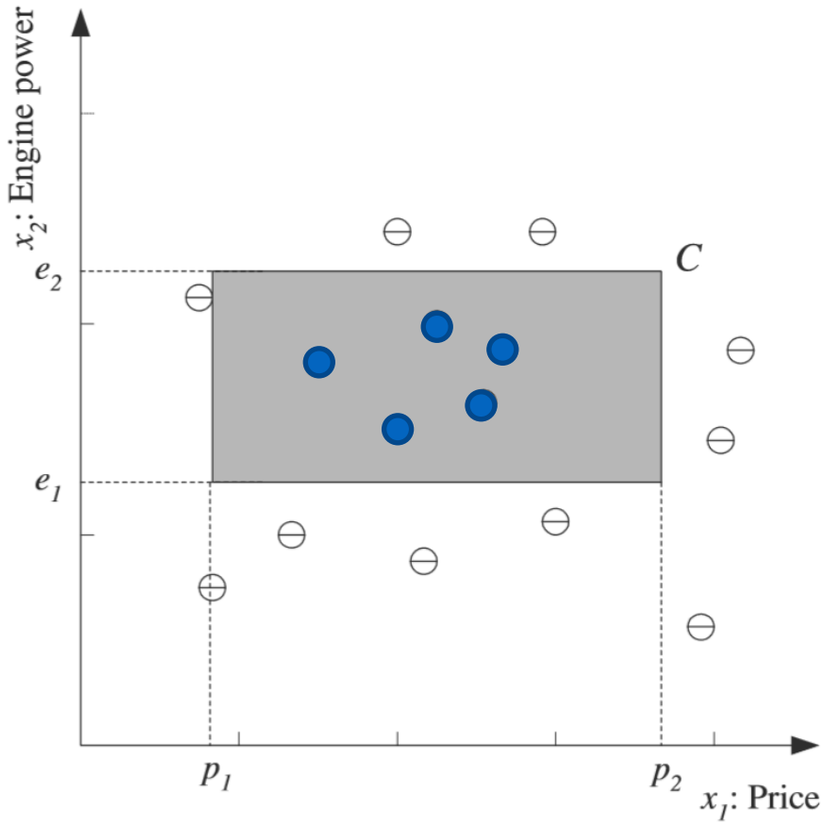
Search space *S* is the set of all possible instances.

Class *C* that needs to modelled.

Instances

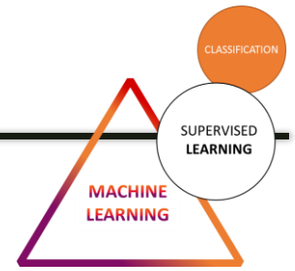


Classes

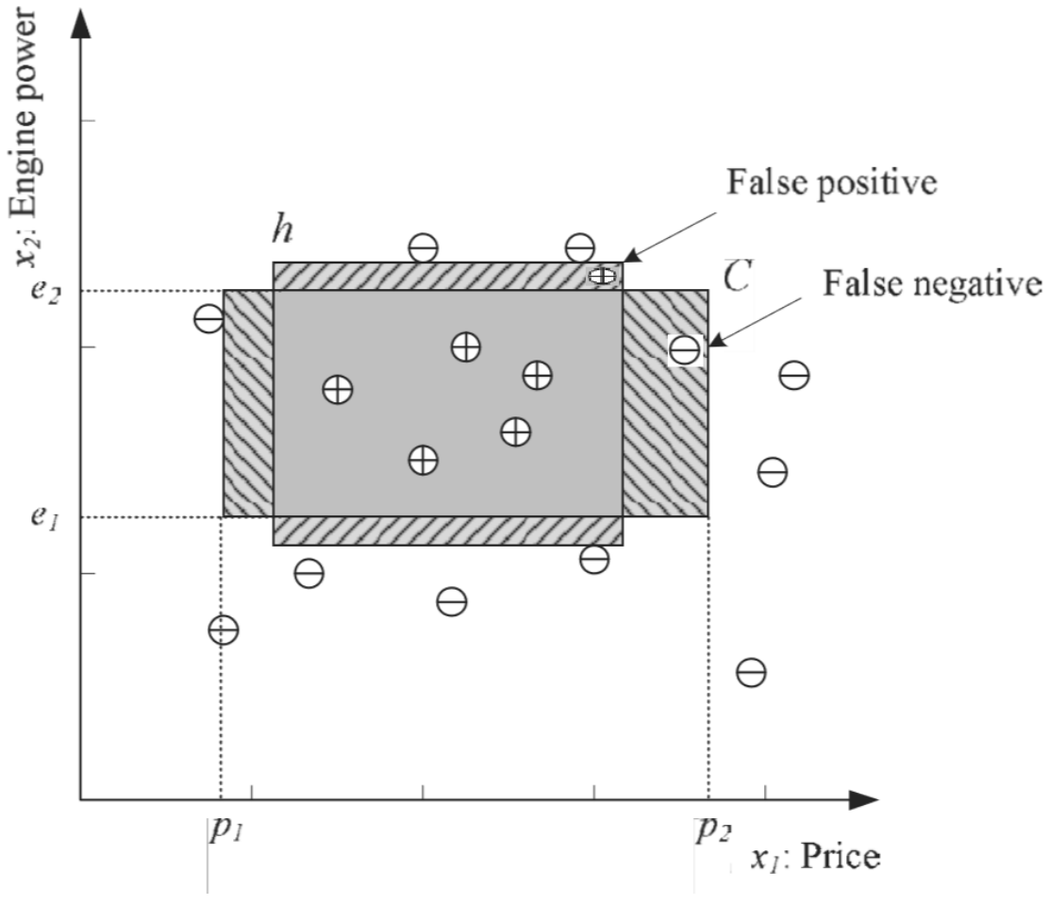


https://mitpress.mit.edu/books/introduction-machine-learning

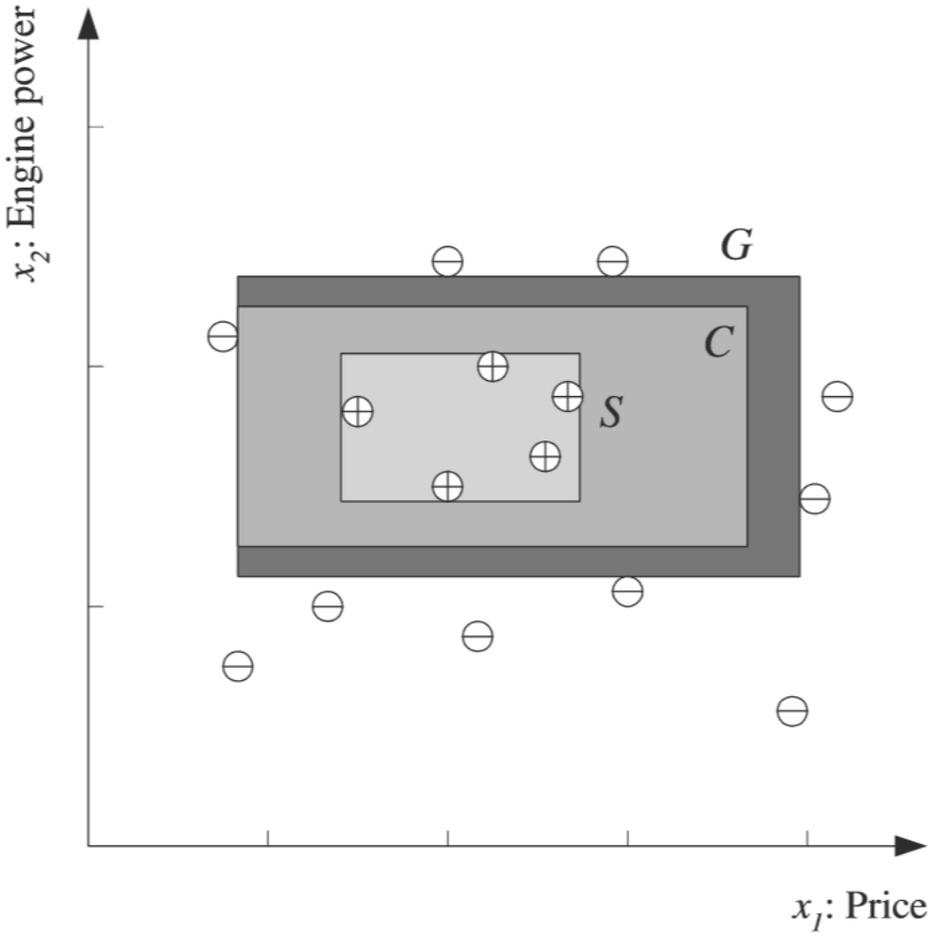
Aspects of classification



Errors

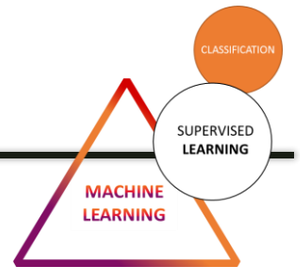


Specific vs General

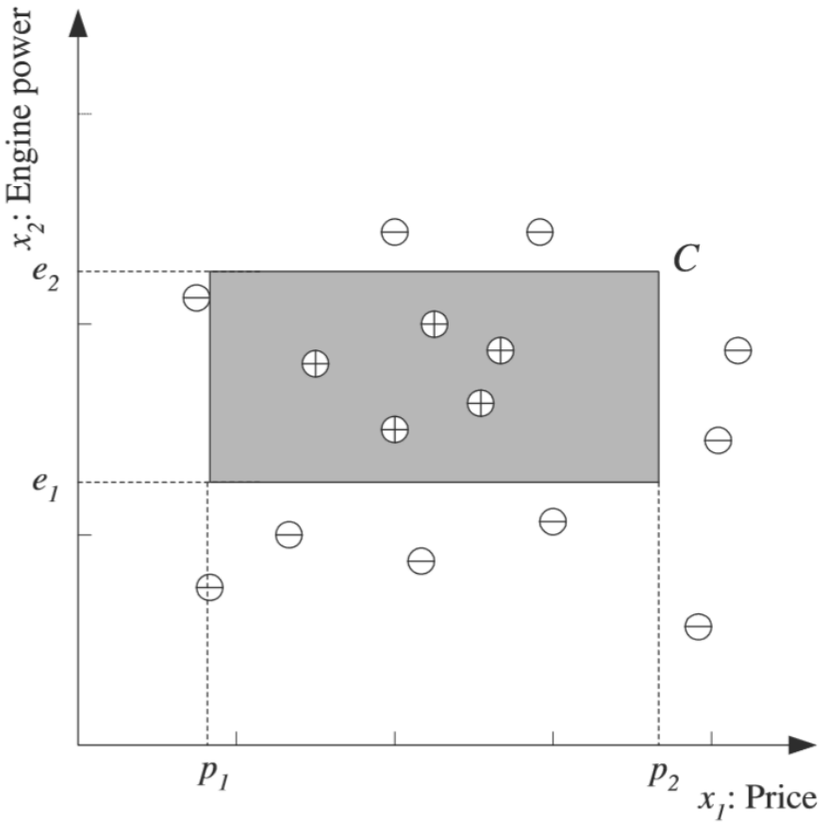


https://mitpress.mit.edu/books/introduction-machine-learning

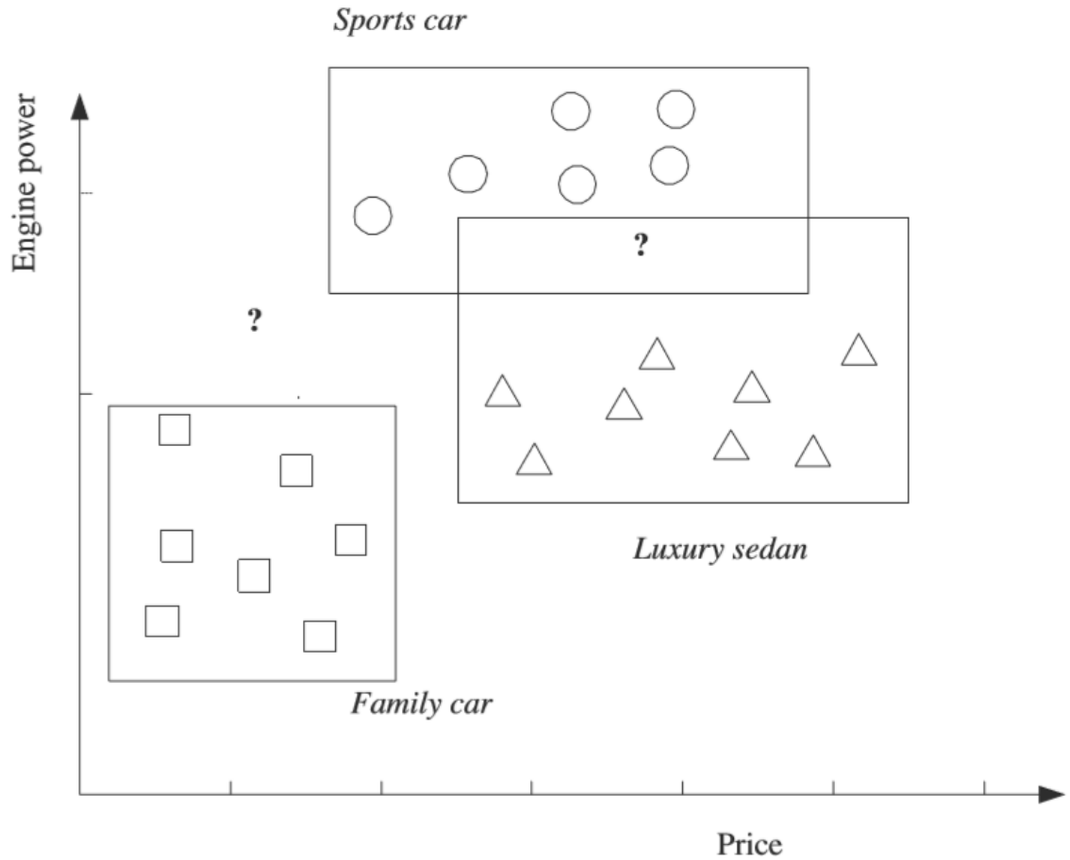
Aspects of classification



Binary

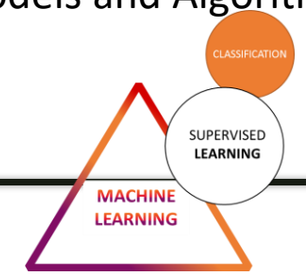


Multiclass



https://mitpress.mit.edu/books/introduction-machine-learning

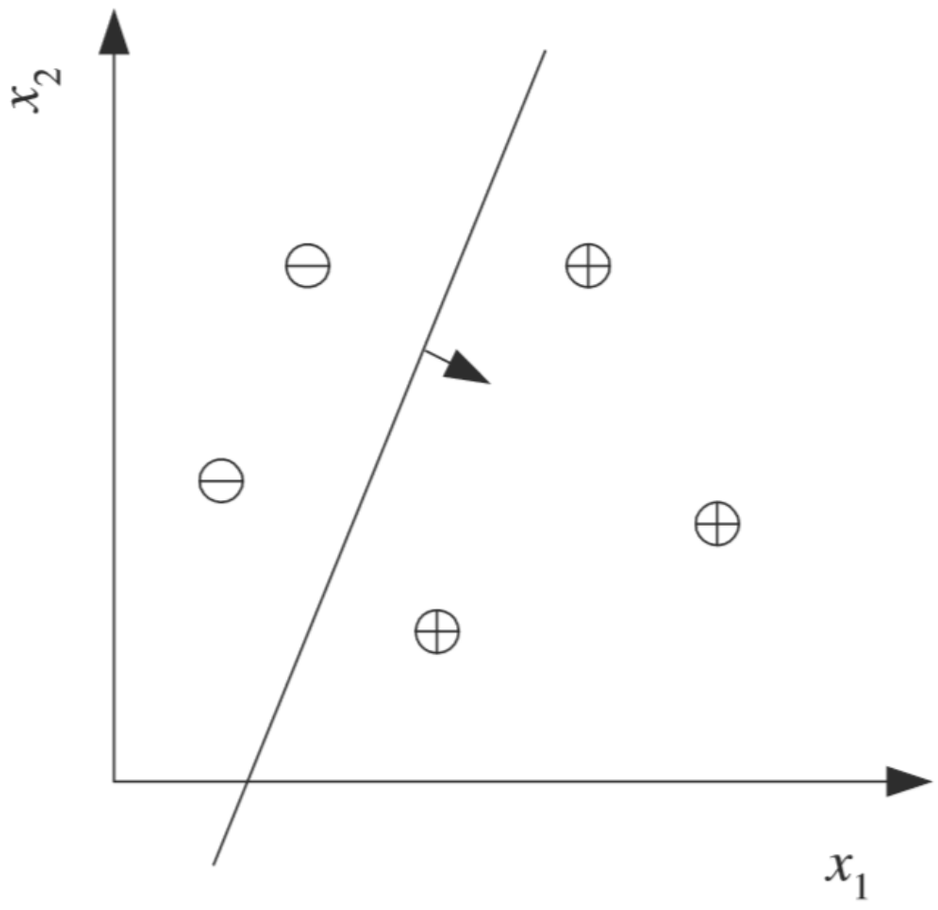
Aspects of Classification



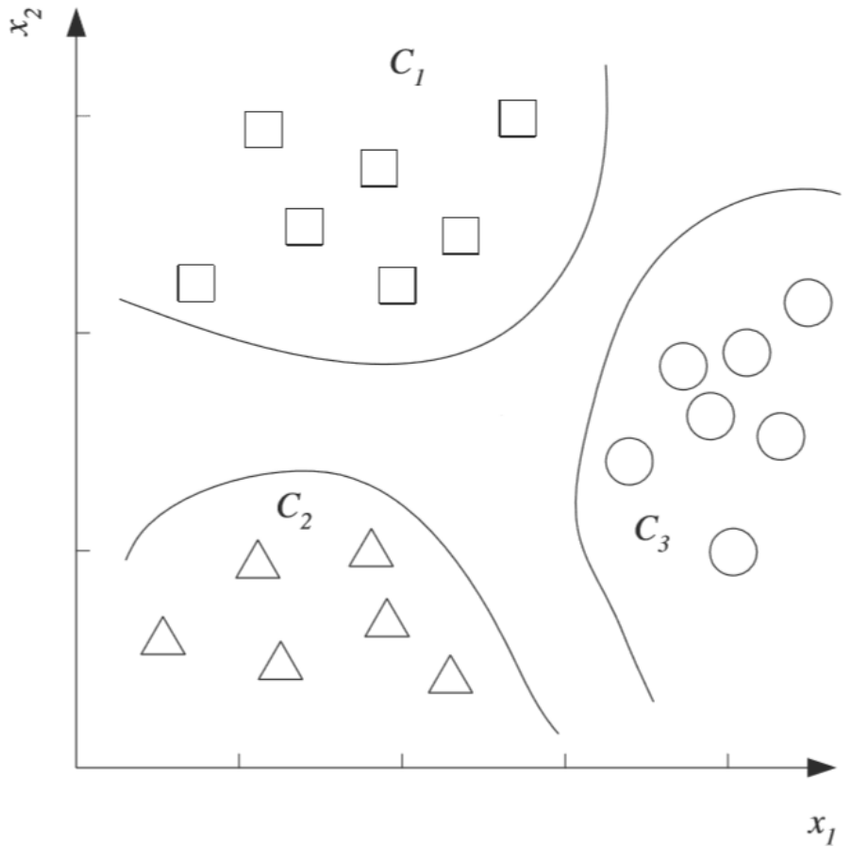
Input: features X_i, *and classes Y*

Output: decision *boundaries* and decision *regions,* amounting to a hypothesis:

Linear

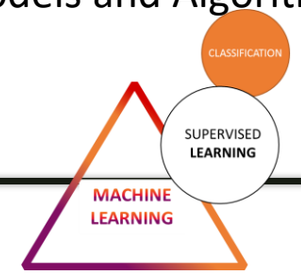


Non-linear



https://mitpress.mit.edu/books/introduction-machine-learning

Models and Algorithms:



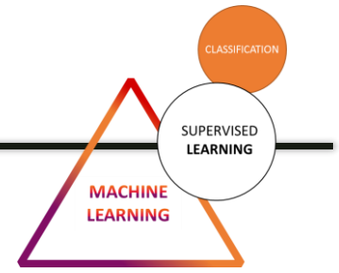
Popular Classifiers

We'll take a (necessarily) quick look at

- KNN
- Perceptrons
- Decision Trees
- SVM
- Boosting
- Random Forests

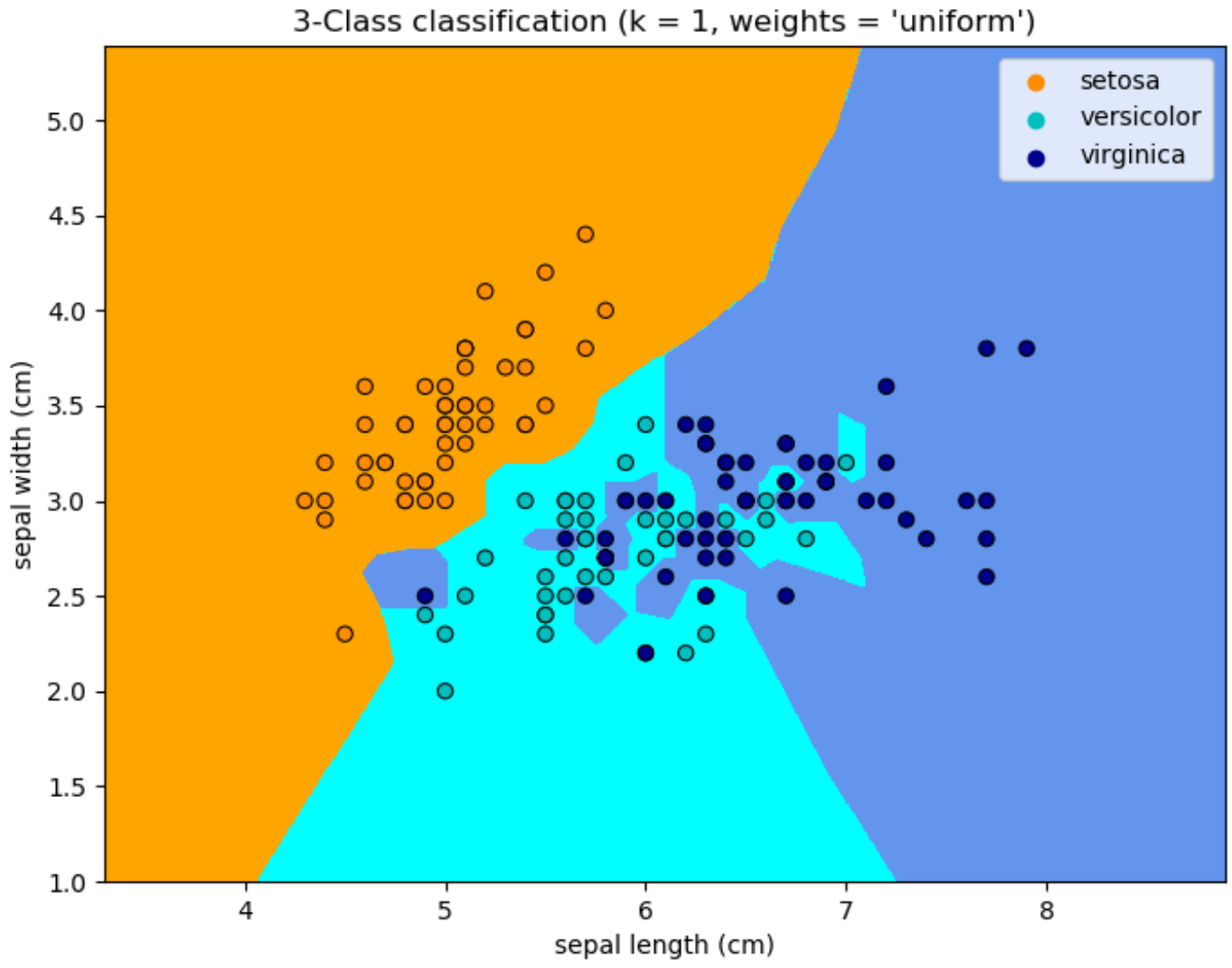
https://scikit-learn.org

k-Nearest-Neighbours classifier



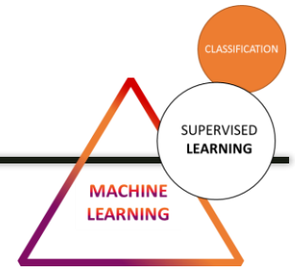
Here is k=1 on the 3-classes “Iris” dataset:

(Voronoi tessellation)

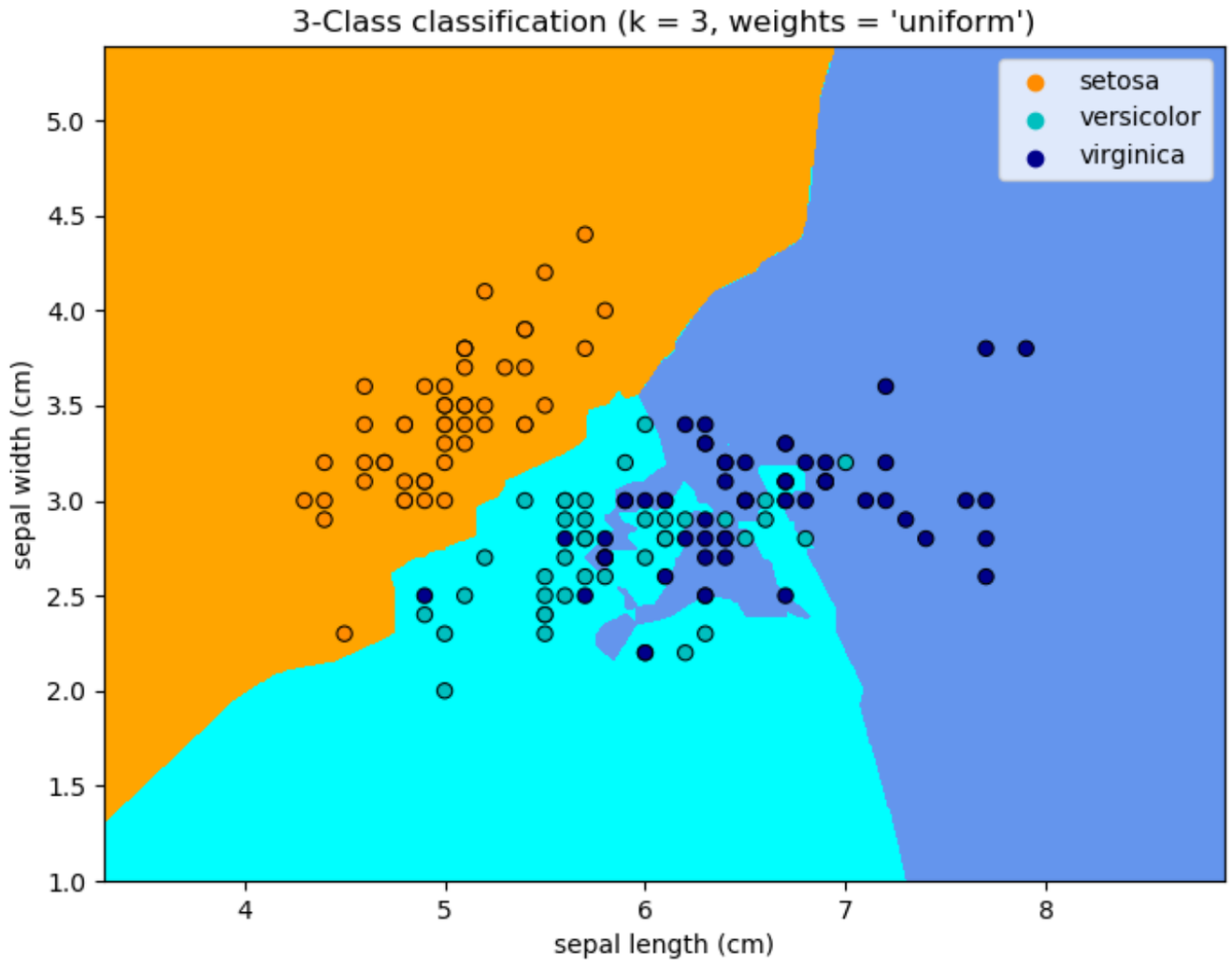


https://scikit-learn.org

k-Nearest-Neighbours classifier

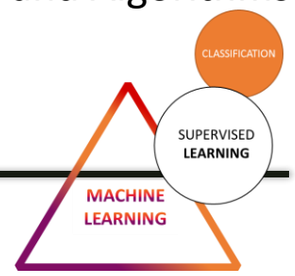


k=3:



https://scikit-learn.org

K-Nearest-Neighbours classifier



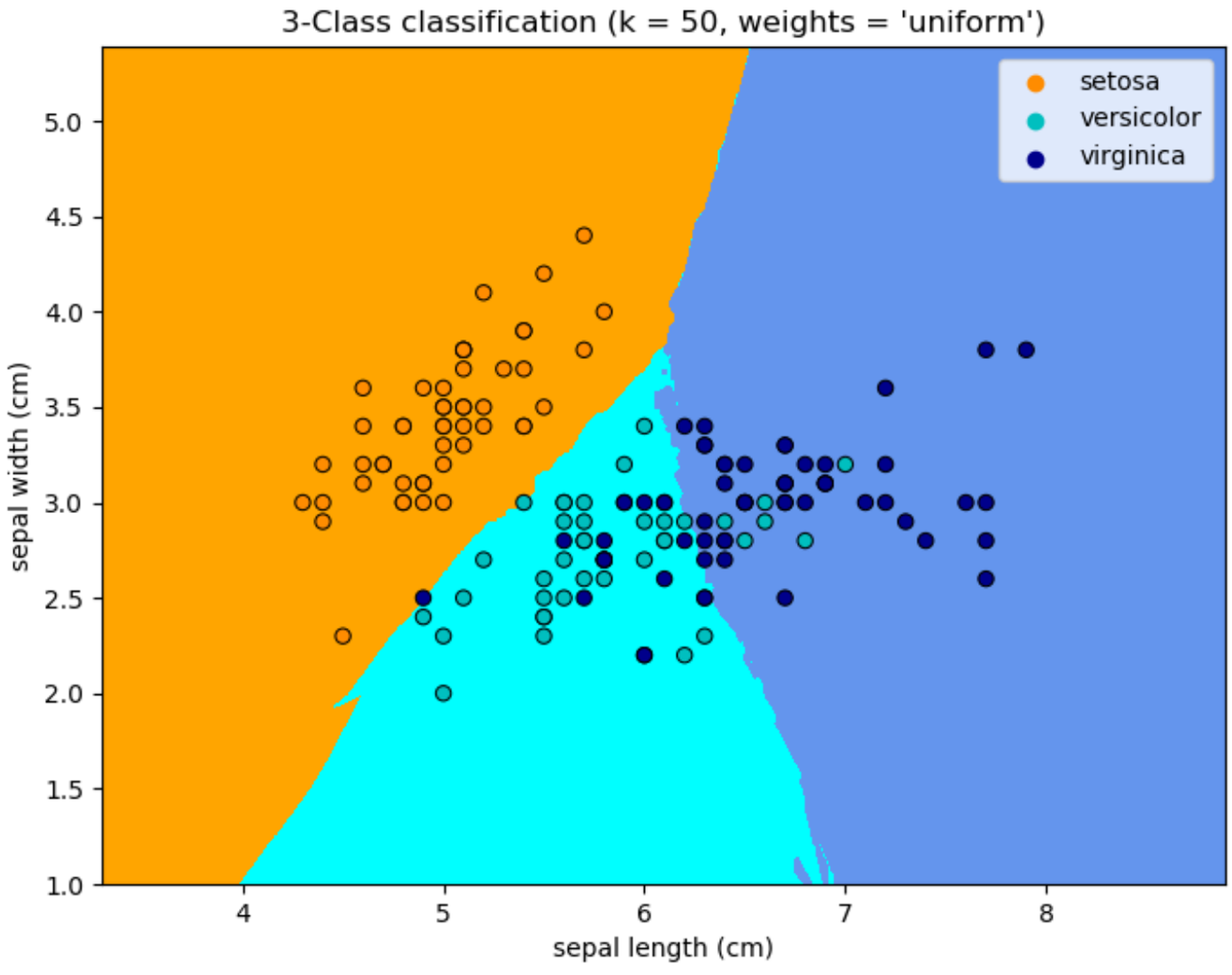
k=50:

hard to beat!

can get slow…

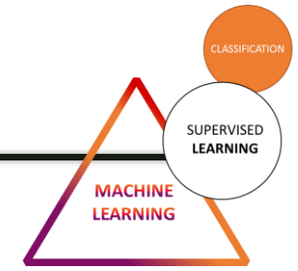
different algorithms for finding the nearest

principle = ?😐



https://scikit-learn.org

Models / algorithms come with options



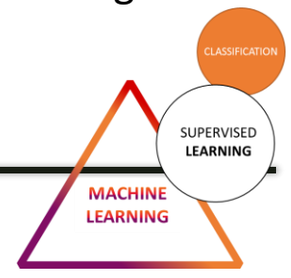
eg: for the k-Nearest-Neighbours ('kNN') classifier

- k
- "nearest" in what sense? (model)
- "nearest" how to find it fast (algorithm)

complexity controls?

- k
  - low → just grabs the nearest example, could over-fit
  - high → averages over lots of nearby examples, could under-fit

Tuning a method vs Evaluating it



eg: what k should I use, in k-Nearest-Neighbours?

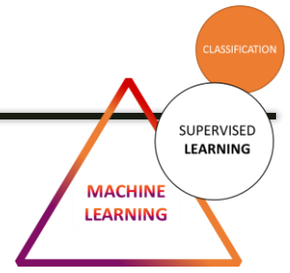
- k=1 will work best on the data it is trained on
- but that's not the point…! how will it do on *other* data? Estimate the worth of the model, e.g. accuracy (% correct)
- Split our data into *Training* set and *"held out"* set (a Test set)

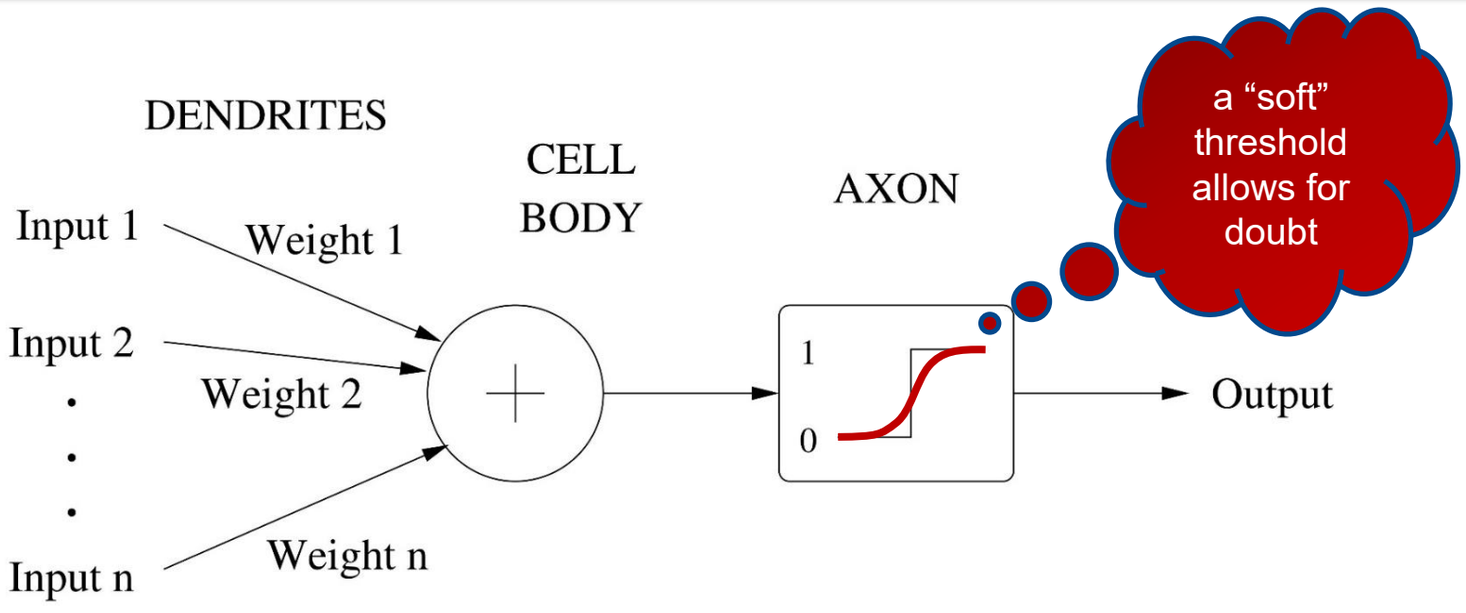
eg. how well can I expect this work in the real world?

- "just use the one with highest Validation Set accuracy?"
- Strangely – no!
- Split our data into *Training+Validation* and *Test* set

You will hear more about this later in the course

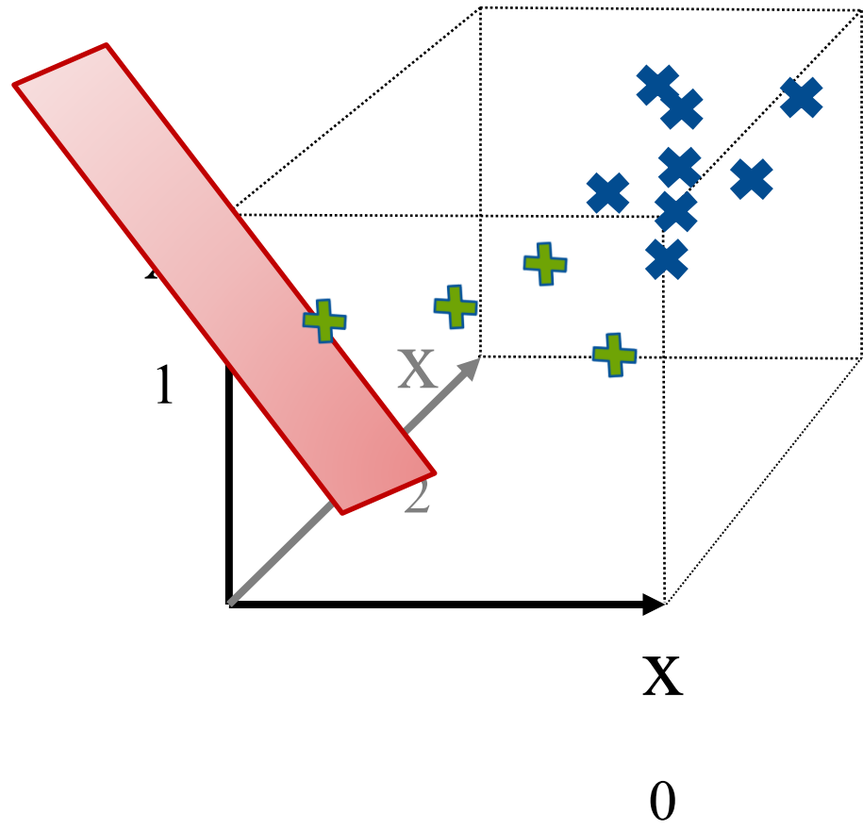
Perceptron classifier (Logistic Regression)



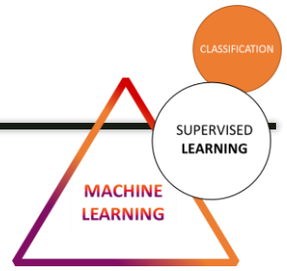


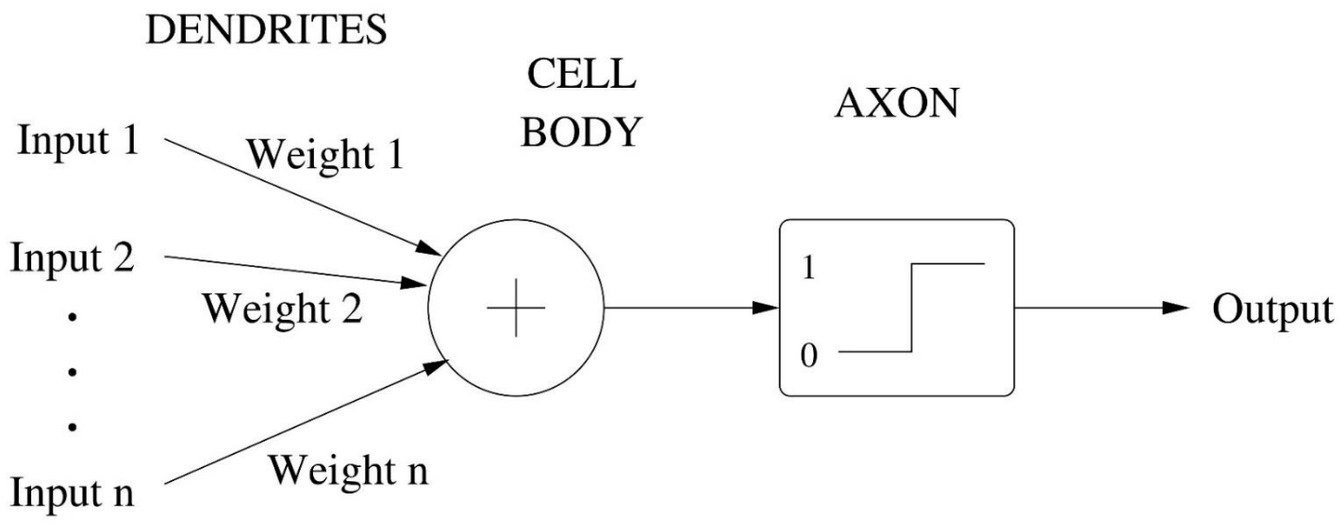
This rule corresponds to drawing a hyperplane in the space of input vectors

- also known as `logistic regression’
- several options for training (minimizing the number of errors on training set)
- “learning” *changes* the weights, which *moves* the hyperplane



Perceptron classifier (Logistic Regression)

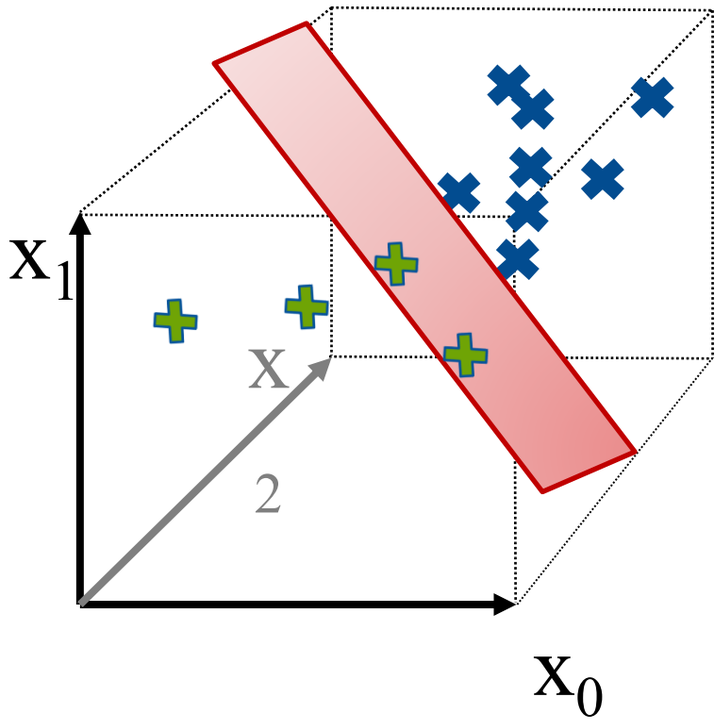




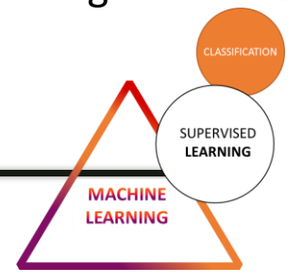
Complexity control - often called "regularisation" is provided by a penalty on the size of weights. Forces Learner to *trade off* errors *vs* big weights.

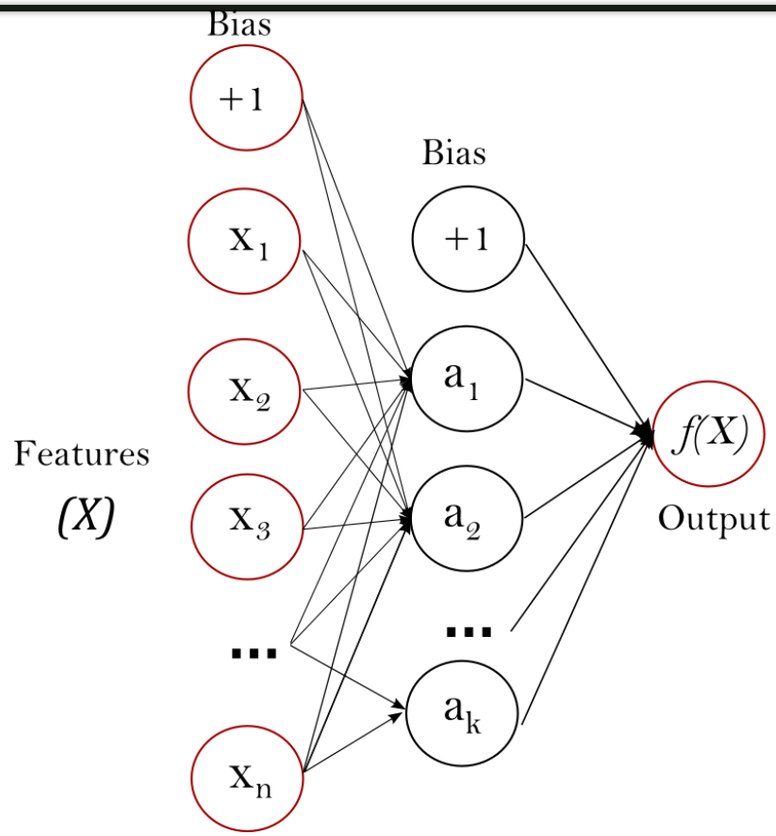
scikit-learn – LogisticRegression calls this "C":

- `Inverse of regularization strength; must be a positive float.
- Smaller values specify stronger regularization.'



Multi-layer Perceptron (MLP)



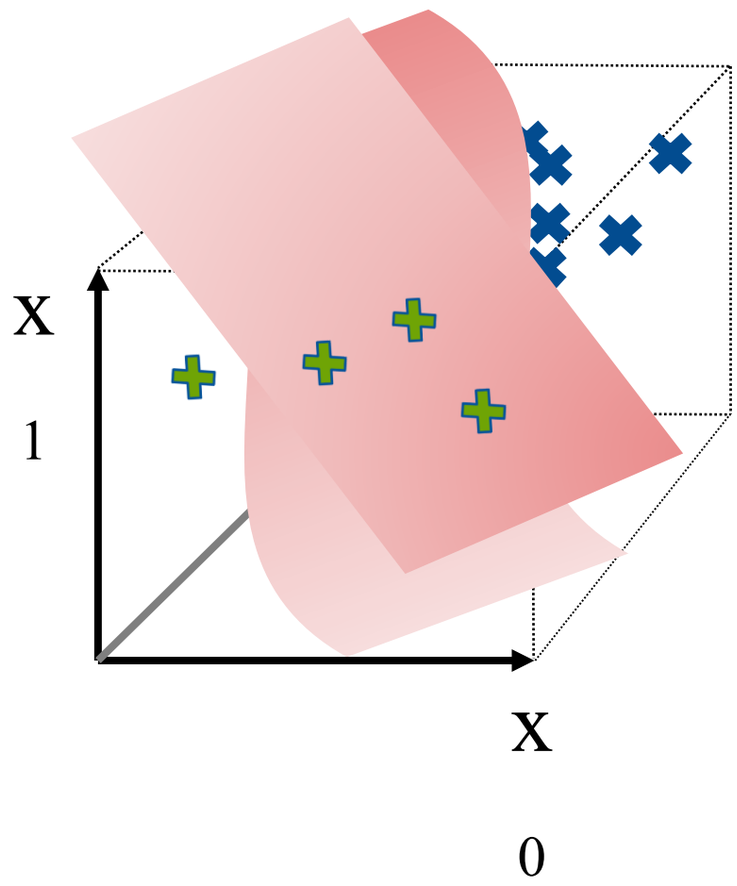


We can chain Perceptrons together, forming a neural network

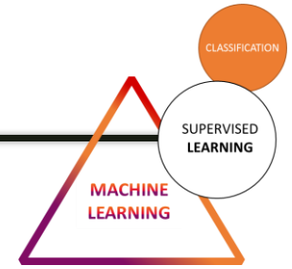
This generalises the hyperplane decision boundary in the space of input vectors into … a more complicated boundary

Typical complexity control: introduce a penalty (or regularisation term), to encourage the weight values to stay "small" (means a *smoother* boundary)

scikit-learn calls this "alpha" - strength of the L2 regularisation term.



Naïve Bayes classifier

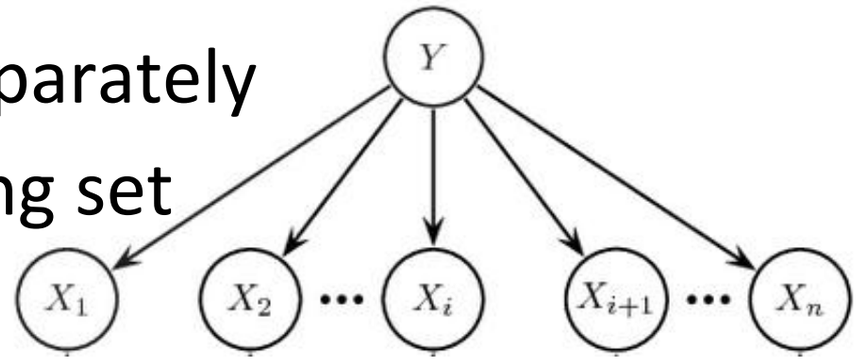


We assume a generative model: imagine class Y *makes* X !

- Naïve Bayes assumption:
  Y makes each X_i *independently* of the others
- this makes the maths, and the learning, easy

$$P(y \mid x_1, \ldots, x_n) = \frac{P(y)P(x_1, \ldots, x_n \mid y)}{P(x_1, \ldots, x_n)} \Rightarrow P(y \mid x_1, \ldots, x_n) = \frac{P(y)\prod_{i=1}^{n} P(x_i \mid y)}{P(x_1, \ldots, x_n)}$$

- we specify *Prob(X_i given Y),* for each *i* separately
- set those probabilities based on the training set
- then at test time, we use Bayes Theorem
  to find *Prob(Y given the new X)*



- seems over-simplified, but work well for many applications

  e.g. document classification and spam filtering.

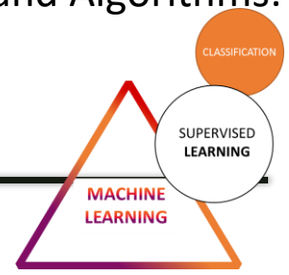
2 flavours:

✓ **Multinomial** Naïve Bayes if X_i are categories

✓ **Gaussian** Naïve Bayes if X_i are real (floats)

https://scikit-learn.org/stable/modules/naive_bayes.html

Naïve Bayes classifier



The multinomial case has a natural “smoothing parameter”,

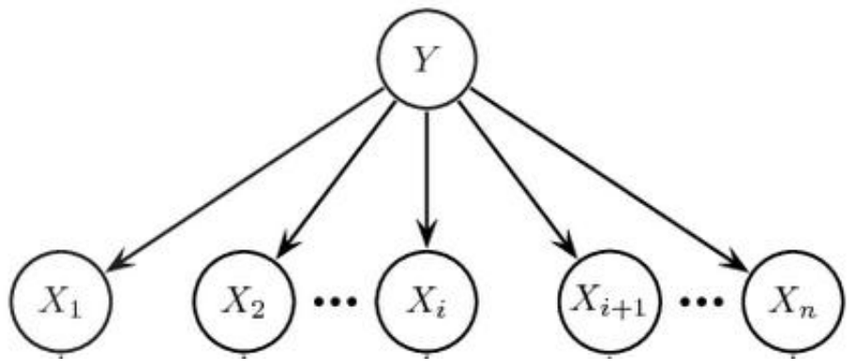
often called a pseudo-count, or Laplace smoothing, used to prevent over-fitting for cases that have zero counts.

The Gaussian case fits a Gaussian for each *Prob(X_i given Y)*.

Complexity control:

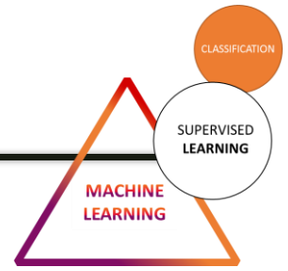
With *very* low counts (e.g. 1) this would over-fit, so we apply “smoothing” (a lower limit) to the variance. If counts are >> 1, this won’t make much difference

Scikit-learn calls this var_smoothing



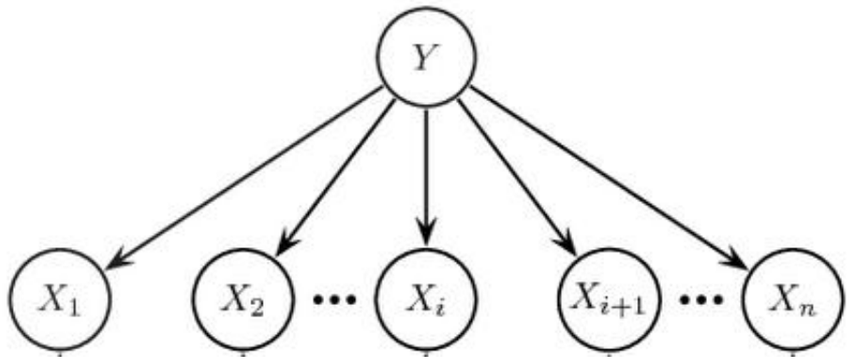
https://scikit-learn.org/stable/modules/naive_bayes.html

Naïve Bayes classifier



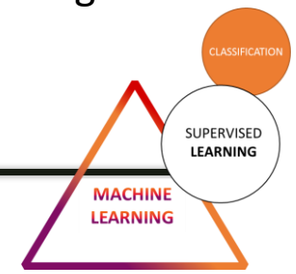
Remarkable factoid:

- the decision surface ends up a hyperplane, just like a perceptron!
- so different models with the same decision surface… huh?
- the two models will place the hyperplane differently



https://scikit-learn.org/stable/modules/naive_bayes.html

Decision Tree classifier



e.g.

Consider V2, is it > 5.1?

if yes: Consider V1, is it < 2.4?

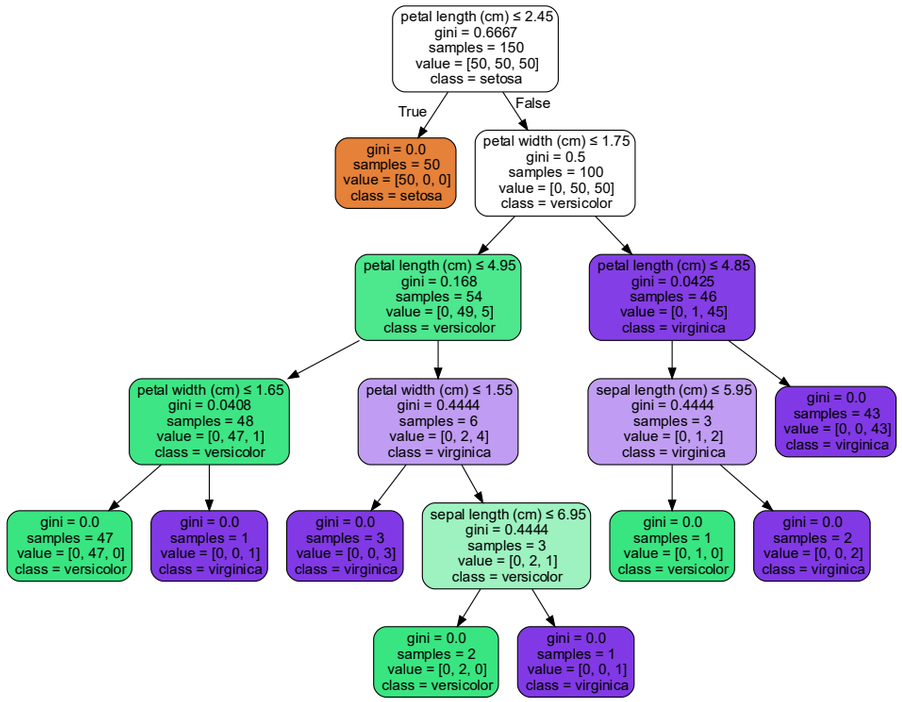
if no: Consider V4, is it > 0.3?

V1	V2	V3	V4	Class
3.6216	8.6661	-2.8073	-0.44699	1
4.5459	8.1674	-2.4586	-1.4621	1
3.866	-2.6383	1.9242	0.10645	1
3.4566	9.5228	-4.0112	-3.5944	1
0.32924	-4.4552	4.57	-0.9888	1
4.3684	9.6718	3.[illegible]06	-3.1625	1
3.5912	3.0129	0.[illegible]888	0.56421	
2.0922	-6.81	8.[illegible]36	-0.60216	
3.2032	5.7588	-0.753[illegible]	-0.61251	
1.5356	9.1772	-2.2718	-0.73535	1
1.2247	8.7779	-2.2135	-0.80647	1

X → Y

And recurse… → tree of decisions.

At the leaves of this tree,
arrive at a classification



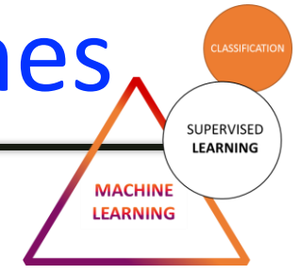
We can learn the tree, namely:

- which variables to consider, in order
- the thresholds

complexity control: scikit-learn suggests the max_depth

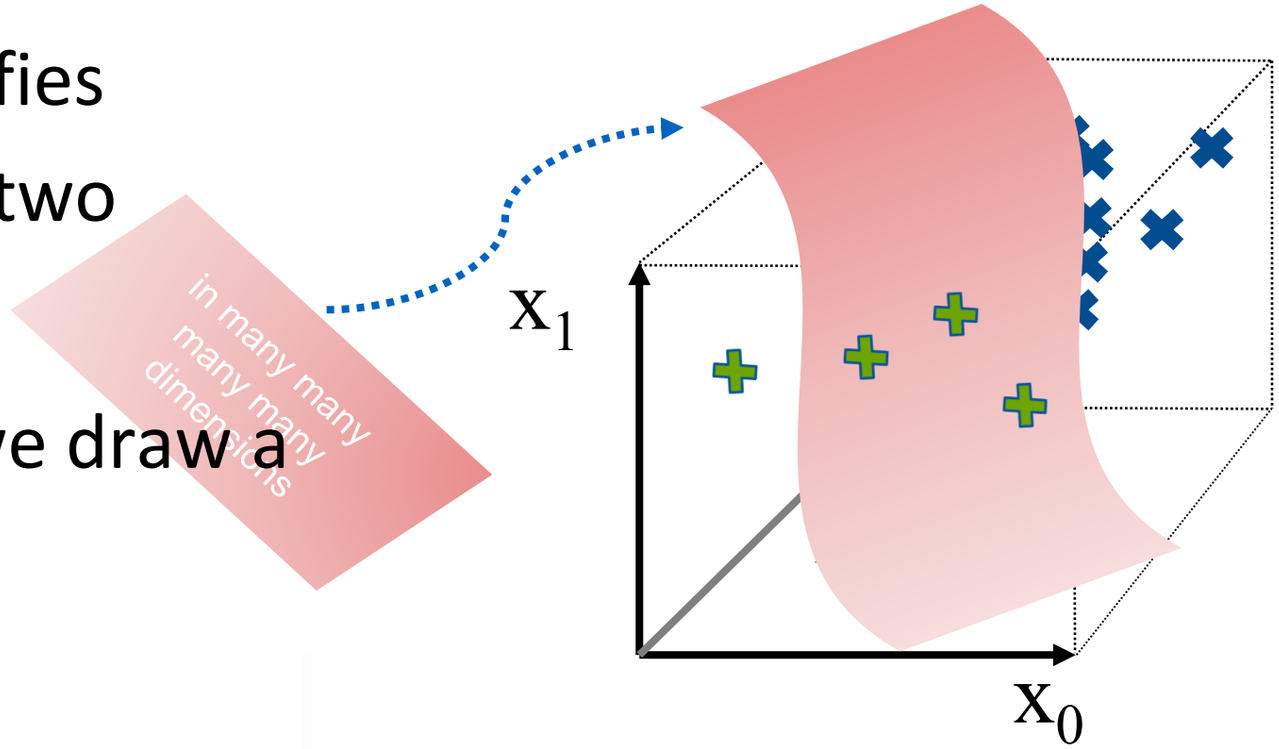
https://docs.daft-pgm.org

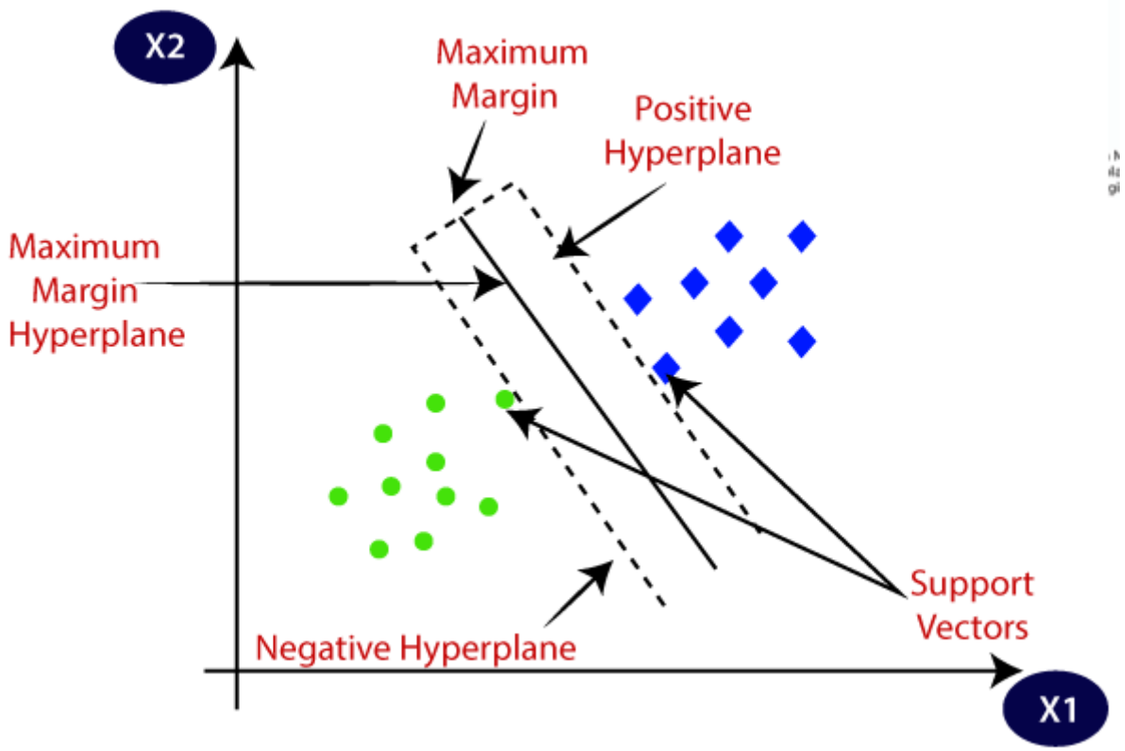
Support Vector Machines / Kernel machines



Two ways to think about SVMs:

- like k-NN, we have a function ("kernel") that quantifies "closeness" between two example X's
- like the Perceptron, we draw a hyperplane in a space

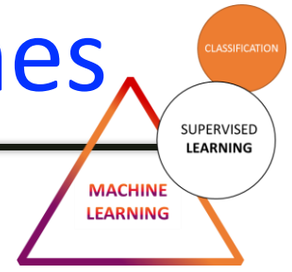




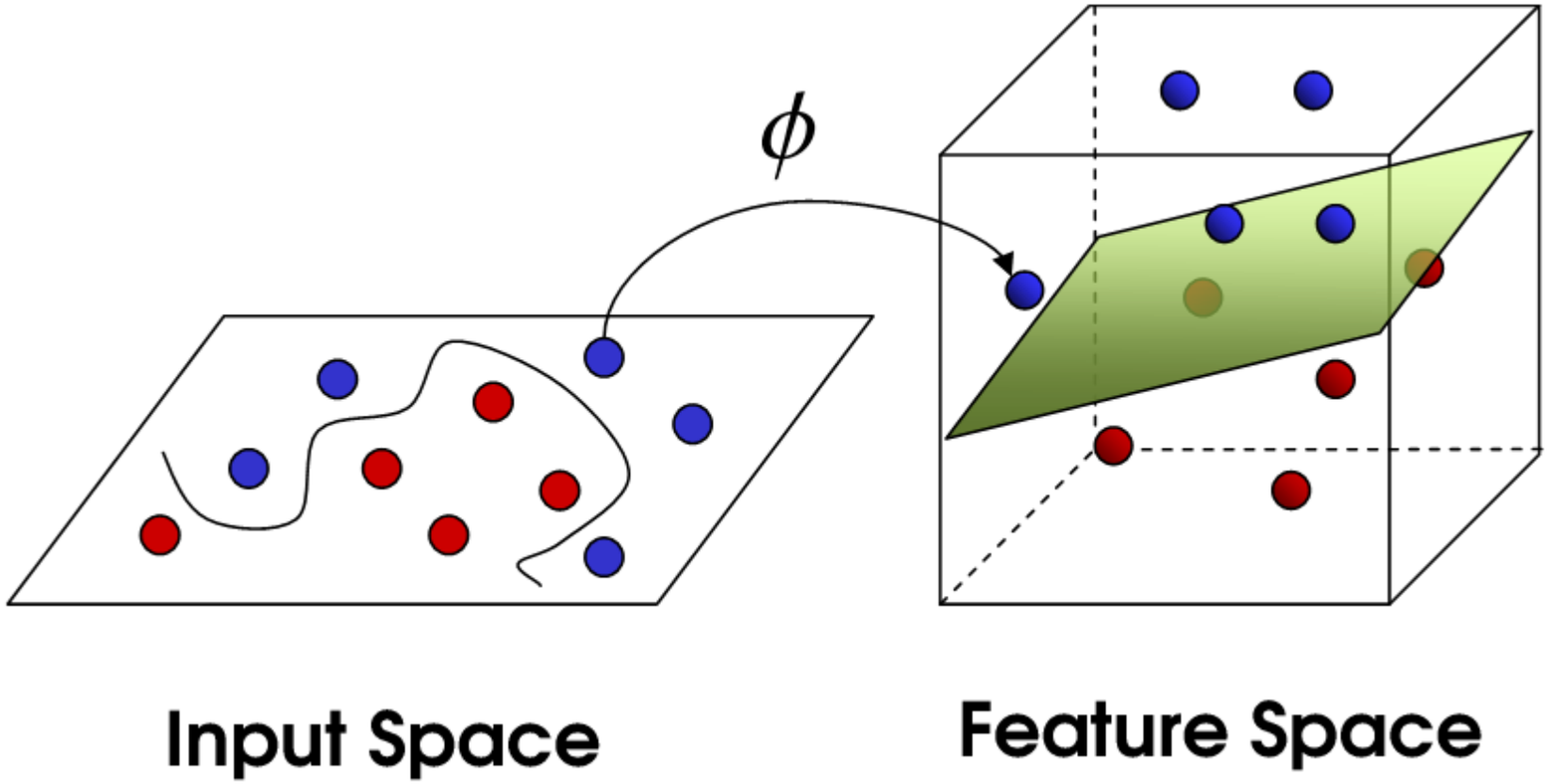
But using math trickery, this very-high-dim "space" is *never explicitly realised*, and the hyperplane is never *explicitly* computed!

https://static.javatpoint.com/tutorial/machine-learning/images/support-vector-machine-algorithm.png

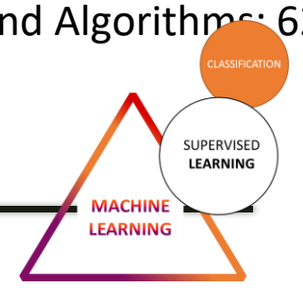
Support Vector Machines / Kernel machines



- Very successful
- Largely robust to choice of kernel function
- Kernel trick - decision boundary can be *linear* in high-dimensional space (nice math properties), & *non-linear* in the input space



Ensembles of classifiers



Several of the most popular and successful classifiers are based on *ensembles* of simple classifiers.

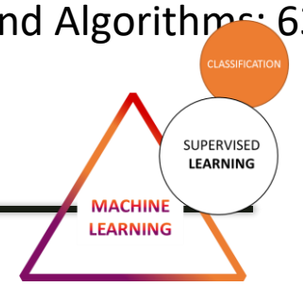
For example:

- Boosting (e.g. GradBoost)
- Random Forests (e.g. of trees / stumps)

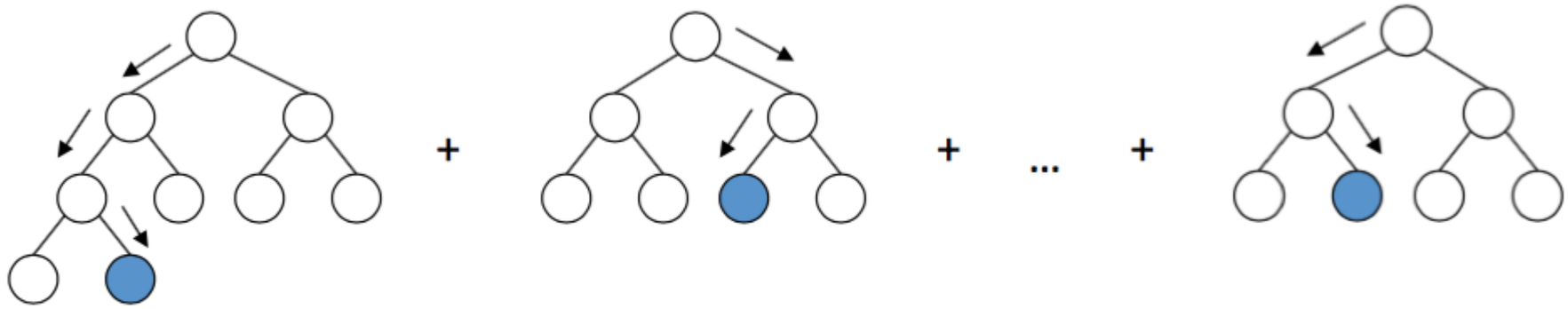
- take hundreds of “base” classifiers which might each be very weak (e.g. “decision stumps”, i.e. one-level decision trees)
- pull these simple predictions together to obtain an ensemble prediction.

complexity control: as with Decision Trees – the max_depth

Gradient Boosted Decision Tree



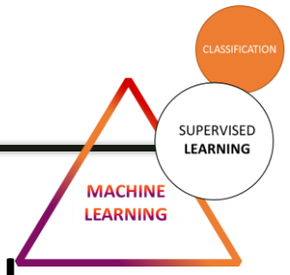
- gradient boosting can create an ensemble of various estimators
- often use gradient boosting over decision trees



- builds an ensemble of DTs one-by-one
- the predictions of the individual trees are summed
- next DT tries to cover the difference between the target value and the current ensemble prediction

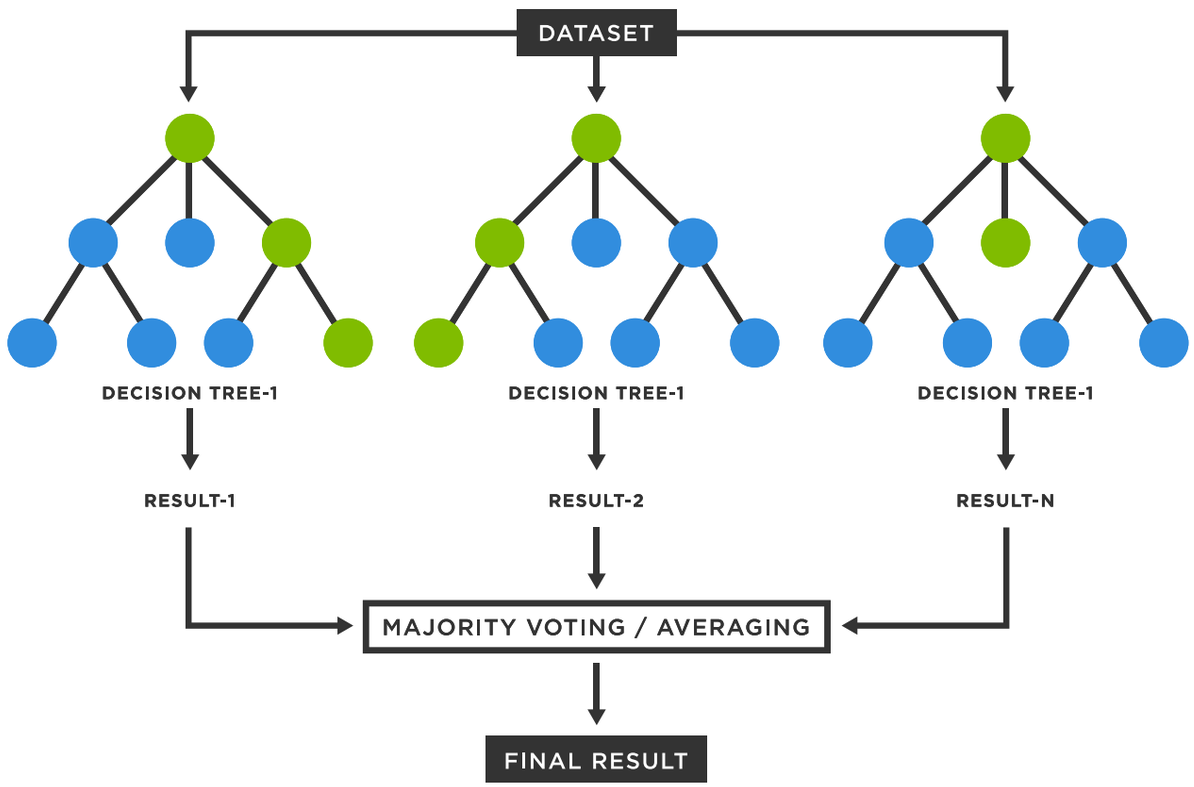
https://arogozhnikov.github.io/2016/06/24/gradient_boosting_explained.html

Random Forests



- each tree is built from a bootstrap sample (i.e. a sample drawn with replacement) from the training set

- when splitting each node during the construction of a tree, the best split is found either from all input features or a random subset of size



https://www.tibco.com/reference-center/what-is-a-random-forest